

Unit 1: Foundations for Systems Development

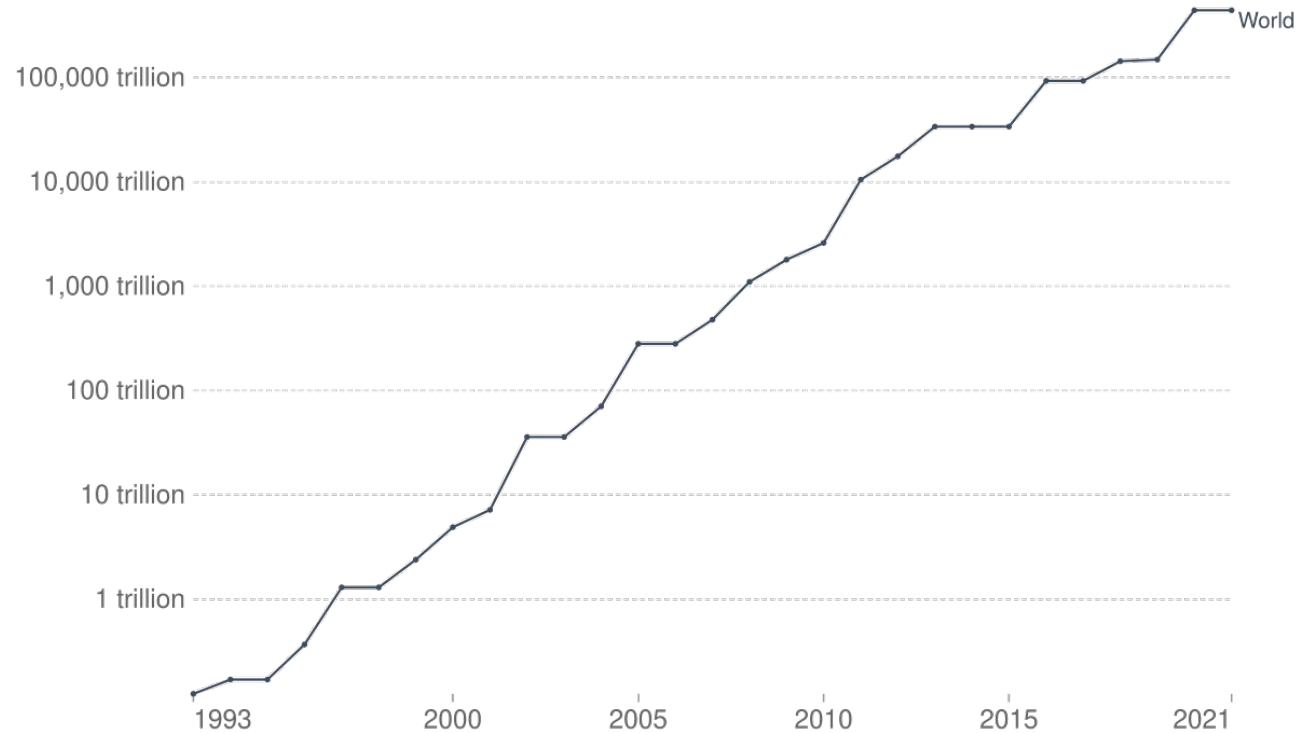
Dr. Sunil Chaudhary
mail4sunil.nep@gmail.com

Agenda

- 1.1. The Systems Development Environment
 - Introduction
 - A Modern Approach to Systems Analysis and Design
 - Developing Information Systems and the Systems Development Life Cycle
 - The Heart of the Systems Development Process and Traditional Waterfall SDLC
 - CASE Tools
- 1.2. Other Approaches
 - Prototyping;
 - Spiral;
 - Rapid Application Development;
 - Introduction to Agile Development
- 1.3. Managing the Information Systems Project
 - Introduction
 - Managing the Information Systems Project
 - Representing and Scheduling Project Plans
 - Using Project Management Software

The exponential growth of supercomputers

Number of floating-point operations carried out per second by the largest supercomputer in any given year



Source: TOP500 Supercomputer Database

Note: Floating-point operations are needed for very large or very small real numbers, or computations that require a large dynamic range. Floating-point operations per second are therefore a more accurate measure than instructions per second.

CC BY

**TRUE OR NOT**

Friday, 06 Oct 2023

8:00 AM MYT

In comparison, the Qualcomm Snapdragon 855 chip used by most flagship Android phones in 2019 boasts 2.84 Gigahertz of processing power.

For those of you who need a refresher, Mega- means 1,000,000 while Giga- is a whopping 1,000,000,000!

According to ZME Science, even a humble graphing calculator had more processing power than the Apollo-era supercomputer.

As physicist Dr Michio Kaku aptly put it, "Today, your cell phone has more computer power than all of NASA back in 1969, when it placed two astronauts on the moon."

Even a much more recent machine – IBM's Deep Blue supercomputer, famed for beating world chess champion Garry Kasparov in 1997 – can't measure up to older-generation smartphones, let alone the latest models.

Of course, both smartphones and supercomputers have developed exponentially since then.

Given these trends, it is not unreasonable to assume that the performance

Sizing up —

Comparing a 1980s supercomputer to a modern smartphone



CRAY-2
supercomputer

5,500 pounds

iPhone 12

5.78 ounces

iPhone 12
1.6 square inches

CRAY-2
supercomputer
16 square feet

If the CRAY-2 were as powerful as the iPhone 12, it would take up a whopping **80,000 sqft** and weigh **27.5 million pounds**.

Power up —

From space landings
to supercomputers
to smartphones

Apollo 11
Guidance Computer



12,250 FLOPS*

CRAY-2
1980s Supercomputer



1.9 billion FLOPS*

Approximately
155,000 times faster
than the Apollo 11
guidance computer

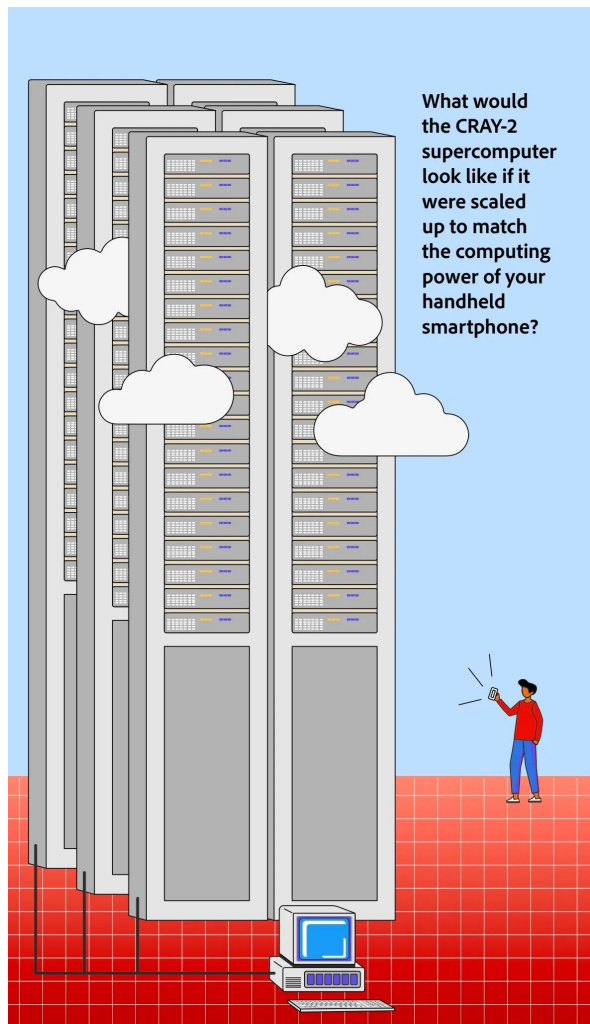
Apple iPhone 12
Smartphone



11 trillion FLOPS*

Approximately **5,000 times faster**
than the CRAY-2 supercomputer
and about **900 million times**
faster than the Apollo 11
guidance computer.

FLOPS: floating point operations per second



What would
the CRAY-2
supercomputer
look like if it
were scaled
up to match
the computing
power of your
handheld
smartphone?

Scene at MIT: Margaret Hamilton's Apollo code

A brief history of the famous 1969 photo of the software that sent humans to the moon.

Maia Weinstock | MIT News Office

August 17, 2016



System Development Environment

Introduction (1/6)

- Information systems analysis and design is the **complex, challenging, and stimulating organizational process** that a team of business and systems professionals uses to develop and maintain information systems. Although advances in information technology continually give us new capabilities, the analysis and design of information systems is driven from an **organizational perspective**.
 - An organization might consist of a whole enterprise, specific departments, or individual work groups. **Organizations can respond to and anticipate problems and opportunities through innovative use of information technology.** Information systems analysis and design is therefore an organizational improvement process.
 - **Systems are built and rebuilt for organizational benefits.** Benefits result from adding value during the process of creating, producing, and supporting the organization's products and services.
- Thus, the analysis and design of information systems **is based on our understanding of the organization's objectives, structure, and processes, as well as your knowledge of how to exploit information technology for advantage.**

Introduction (2/6)

- **Information systems support almost everything organizations do,** whether the systems are developed for internal use, for exchanges with business partners, or for interactions with customers.
 - Networks, especially the Internet—especially the World Wide Web—are crucial for connecting organizations with their partners and their customers. The overwhelming majority of business use of the Web is business-to-business applications.
- **Regardless of the technology involved, understanding the business and how it functions is the key to successful systems analysis and design,** even in the fast-paced, technology-driven environment that organizations find themselves in today.

Introduction (3/6)

- **Analyzing and designing information systems** will give us the **chance to understand organizations at a depth and breadth.**
- An important (but not the only) **result of systems analysis and design** is **application software**, software designed to support a specific organizational function or process.
- In addition to application software, the total **information system** includes the **hardware and systems software** on which the application software runs, **documentation and training materials**, the **specific job roles associated with the overall system**, **controls**, and the **people who use the software** along with their work methods.
- Although we will address all of these various dimensions of the overall system, we will emphasize **application software development—our primary responsibility as a systems analyst.**

Introduction (4/6)

- In the early years of computing, **analysis and design were considered an art or a craft**. Rapid growth in the need for systems in the 1970s resulted in a **highly structured approach to systems analysis and design**.
- While the structured approach is still in use, **current approaches focus on rapid and constant software delivery, managed by small teams of talented developers**.
 - This approach, called **agile development**, has become standard for most organizations that develop systems. In fact, 94% of companies report that they practice agile in their systems development efforts.

Introduction (5/6)

- **Methodologies** are comprehensive, multiple-step approaches to systems development that will guide your work and influence the quality of your final product—the information system.
 - A methodology adopted by an organization will be **consistent with its general management style** (e.g., an organization's orientation toward consensus management will influence its choice of systems development methodology). Most methodologies incorporate several development techniques.
- **Techniques** are particular processes that we, as an analyst, will follow to help ensure that our work is well thought out, complete, and comprehensible to others on your project team.
 - Techniques **provide support for a wide range of tasks**, including gathering information to determine what your system should do, planning and managing the activities in a systems development project, diagramming the system's logic, and designing the system's interface and outputs.
- **Tools** are typically computer programs that make it easy to use and benefit from techniques and to faithfully follow the guidelines of the overall development methodology.
 - To be effective, techniques and tools must **both be consistent with an organization's systems development methodology**. Techniques and tools must make it easy for system developers to conduct the steps called for in the methodology. These three elements—methodologies, techniques, and tools—work together to form an organizational approach to systems analysis and design.

Introduction (6/6)

- Although many people in organizations are responsible for systems analysis and design, in most organizations the **systems analyst has the primary responsibility.**
 - The primary role of a systems analyst is **to study the problems and needs of an organization in order to determine how people, methods, and information technology can best be combined to bring about improvements in the organization.**
 - A systems analyst **helps system users and other business managers define their requirements for new or enhanced information services.**
 - As such, a systems analyst is an agent of change and innovation.

A Modern Approach to Systems Analysis and Design (1/5)

- The analysis and design of computer-based information systems **began in the 1950s**. Since then, the development environment has **changed dramatically, driven by organizational needs as well as by rapid changes in the technological capabilities** of computers.
 - In the 1950s, **development focused on the processes the software performed**. Because computer power was a critical resource, efficiency of processing became the main goal. Computers were large, expensive, and not very reliable.
 - **Emphasis was placed on automating existing processes**, such as purchasing or payroll, often within single departments. **All applications had to be developed in machine language or assembly language**, and they had to be **developed from scratch** because there was no software industry.
 - Because computers were so **expensive**, computer memory was also at a premium, so **system developers conserved as much memory as possible for data storage**.

A Modern Approach to Systems Analysis and Design (2/5)

- The first **procedural or third-generation computer programming languages** did not become available until the beginning of the 1960s.
 - Computers were still large and expensive, but the 1960s saw important breakthroughs in technology that enabled the **development of smaller, faster, less expensive computers—minicomputers—and the beginnings of the software industry.**
 - Most organizations still **developed their applications from scratch using their in-house development staff.**
 - Systems development was **more an art than a science.** This **view of system development began to change in the 1970s,** however, as **organizations started to realize how expensive it was to develop customized information systems for every application.**
 - Systems development **came to be more disciplined as many people worked to make it more like engineering.**
 - Early **database management systems, using hierarchical and network models,** helped bring discipline to the storage and retrieval of data. The development of database management systems helped shift the focus of system development from processes first to data first.

A Modern Approach to Systems Analysis and Design (3/5)

- **The 1980s were marked by major breakthroughs in computing in organizations, as microcomputers became key organizational tools.**
 - The software industry expanded greatly as **more and more people began to write off-the-shelf software for microcomputers.**
 - Developers began to **write more and more applications in fourth-generation languages, which, unlike procedural languages, instructed a computer on what to do instead of how to do it.**
 - **Computer-aided software engineering (CASE) tools were developed** to make systems developers' work easier and more consistent.
 - As **computers continued to get smaller, faster, and cheaper**, and as the **operating systems** for computers **moved away from line prompt interfaces to windows-** and icon-based interfaces, organizations moved to applications with more graphics.
 - **Organizations developed less software in-house and bought relatively more from software vendors.**
 - The systems developer's **job went through a transition from builder to integrator.**

A Modern Approach to Systems Analysis and Design (4/5)

- The systems development environment of the late **1990s focused on systems integration**.
 - Developers used **visual programming environments**, such as Visual Basic, to design the user interfaces for systems that run on client/server platforms.
 - The **database**, which may be **relational or object-oriented** and which may have been developed using software from firms such as Oracle, resided on the server.
 - In many cases, the **application logic resided on the same server**.
 - Alternatively, an organization may have decided to **purchase its entire enterprise-wide system from companies** such as SAP AG or Oracle.
 - **Enterprise-wide systems are large, complex systems** that consist of a series of independent system modules.
 - Developers **assemble systems by choosing and implementing specific modules**.
 - Starting in the middle years of the 1990s, **more and more systems development efforts focused on the Internet, especially the Web**.

A Modern Approach to Systems Analysis and Design (5/5)

- **Today there is continued focus on developing systems for the Internet and for firms' intranets and extranets.**
 - More and more, **system implementation involves a three-tier design**, with the **database on one server**, the **application on a second server**, and **client logic located on user machines**.
 - Another important development is the **move to wireless system components**. Wireless devices can access Web-based applications from almost anywhere.
 - Finally, the **trend continues toward assembling systems from programs and components purchased off the shelf**.
 - In many cases, **organizations do not develop the application in house**.
 - They **don't even run the application in-house**, choosing instead to use the application on a per-use basis by accessing it through the **cloud**.

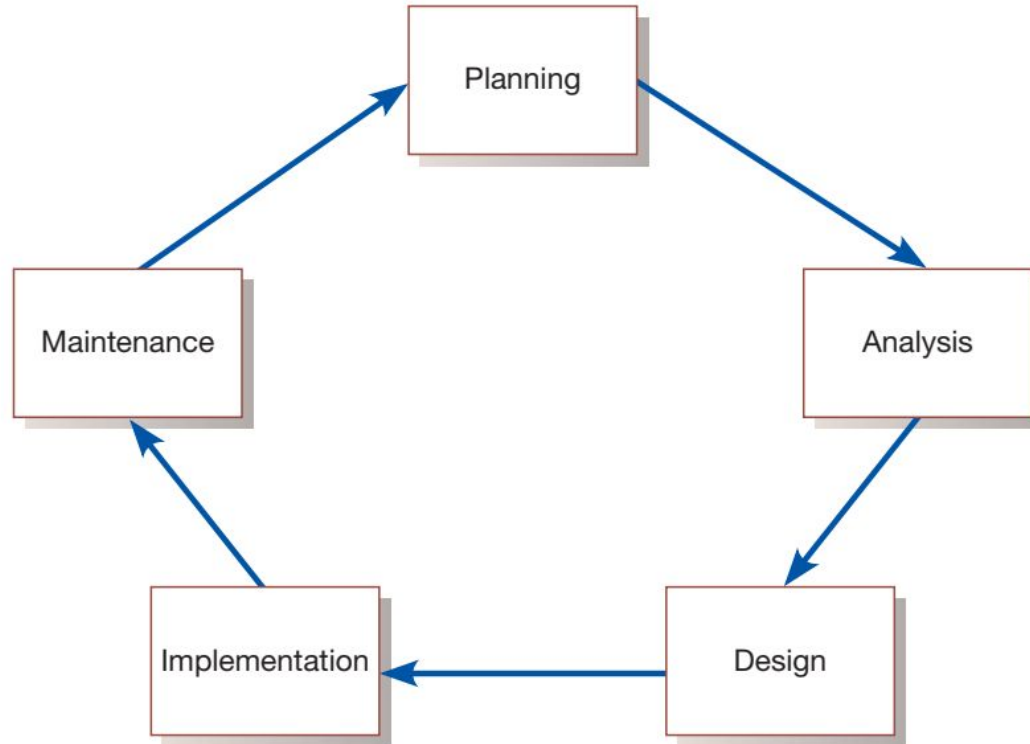
Developing Information Systems and the Systems Development Life Cycle (1/16)

- Whether they rely on **structured or agile approaches**, or on a **hybrid**, most organizations find it **beneficial to use a standard set of steps**, called a **systems development methodology**, to develop and support their information systems.
 - Like many processes, the development of information systems often **follows a life cycle**.
 - For example, a **commercial product follows** a life cycle in that it is **created, tested, and introduced to the market**. Its sales increase, peak, and decline. Finally, the product is removed from the market and replaced by something else.
- The **systems development life cycle (SDLC)** is a common methodology for systems development in many organizations; it **features several phases that mark the progress of the systems analysis and design effort**.
- Every text book author and information systems development organization uses a slightly different life-cycle model, with anywhere from **3 to almost 20 identifiable phases**.

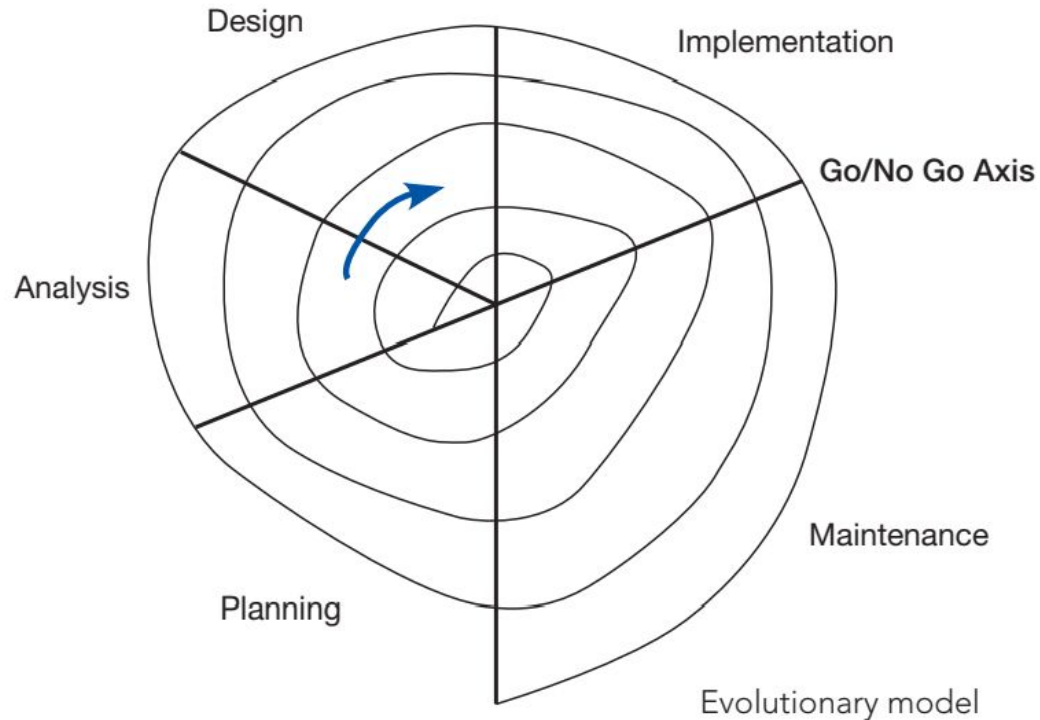
Developing Information Systems and the Systems Development Life Cycle (2/16)

- **The life cycle can be thought of as a circular process in which the end of the useful life of one system leads to the beginning of another project that will develop a new version or replace an existing system altogether.**
 - At first glance, the **life cycle appears to be a sequentially ordered set of phases, but it is not.** The specific **steps and their sequence are meant to be adapted as required for a project, consistent with management approaches.** For example, in any given SDLC phase, the project can return to an earlier phase if necessary. Similarly, if a commercial product does not perform well just after its introduction, it may be temporarily removed from the market and improved before being reintroduced.
 - In the SDLC, it is also possible to **complete some activities in one phase in parallel with some activities of another phase.** Sometimes the **life cycle is iterative**; that is, phases are repeated as required until an acceptable system is found.
 - Some people **consider the life cycle to be a spiral**, in which we constantly cycle through the phases at different levels of detail. **However conceived, the systems development life cycle used in an organization is an orderly set of activities conducted and planned for each development project.**
 - **The skills required of a systems analyst apply to all lifecycle models.**
- **Software is the most obvious end product of the life cycle**; other essential outputs include **documentation** about the system and how it was developed, as well as **training for users.**

Developing Information Systems and the Systems Development Life Cycle (3/16)



Developing Information Systems and the Systems Development Life Cycle (4/16)



Developing Information Systems and the Systems Development Life Cycle (5/16)



Initiation



System Concept Development



Planning



Requirements Analysis



Design



Development



Integration and Test



Implementation



Operation and Maintenance



Disposition

U.S. Department of Justice's systems
development life cycle

Developing Information Systems and the Systems Development Life Cycle (6/16)

- **Every medium-to-large corporation and every custom software producer will have its own specific life cycle or systems development methodology in place.**
 - Even if a particular methodology does not look like a cycle, we will probably discover that many of the SDLC steps are performed and SDLC techniques and tools are used.

Developing Information Systems and the Systems Development Life Cycle (7/16)

- The **first phase** in the SDLC is **planning**. In this phase, **someone identifies the need for a new or enhanced system**.
- In **larger organizations**, this **recognition may be part of a corporate and systems planning process**.
 - **Information needs of the organization as a whole are examined**, and projects to meet these needs are proactively identified.
 - The organization's information system needs may result from requests to deal with problems in current procedures, from the desire to perform additional tasks, or from the realization that information technology could be used to capitalize on an existing opportunity.
 - These **needs can then be prioritized and translated into a plan** for the information systems department, including a **schedule for developing** new major systems.
- In **smaller organizations (as well as in large ones)**, the **determination of which systems to develop may be affected by ad hoc user requests submitted** as the need for new or enhanced systems arises, as well as from a formalized information planning process.
 - In either case, during **project identification and selection**, an **organization determines whether resources should be devoted to the development or enhancement of each information system under consideration**.
 - The **outcome of the project identification and selection process is a determination of which systems development projects should be undertaken by the organization**, at least in terms of an initial study.

Developing Information Systems and the Systems Development Life Cycle (8/16)

- Two **additional major activities** are also performed during the planning phase:
 - the formal yet still preliminary investigation of the system problem or opportunity at hand and
 - the presentation of reasons why the system should or should not be developed by the organization.
- A **critical step** at this point is **determining the scope of the proposed system**.
 - The project leader and initial team of systems analysts also **produce a specific plan for the proposed project the team will follow** using the remaining SDLC steps. This **baseline project plan** customizes the **standardized SDLC** and **specifies the time and resources needed for its execution**.
- The **formal definition** of a project is **based on the likelihood that the organization's information systems department is able to develop a system that will solve the problem or exploit the opportunity and determine whether the costs of developing the system outweigh the benefits it could provide**.
 - The **final presentation of the business case for proceeding** with the subsequent project phases is usually made by the project leader and other team members to **someone in management or to a special management committee** with the job of deciding which projects the organization will undertake.

Developing Information Systems and the Systems Development Life Cycle (9/16)

- The **second phase** in the SDLC is **analysis**. During this phase, **the analyst thoroughly studies the organization's current procedures and the information systems used to perform organizational tasks**.
- Analysis has two subphases.
 - The first is **requirement determination**. In this subphase, analysts work with users to determine what the users want from a proposed system. The requirements determination process usually involves a careful study of any current systems, manual and computerized, that might be replaced or enhanced as part of the project.
 - In the second part of analysis, analysts **study the requirements and structure them according to their interrelationships and eliminate any redundancies**. The output of the analysis phase is a description of (but not a detailed design for) the alternative solution recommended by the analysis team.
- Once the **recommendation is accepted** by those with funding authority, the analysts can **begin to make plans to acquire any hardware and system software necessary to build or operate the system as proposed**.

Developing Information Systems and the Systems Development Life Cycle (10/16)

- The **third phase** in the SDLC is **design**. During design, analysts **convert the description of the recommended alternative solution into logical and then physical system specifications**.
- Analysts **aid in the design of all aspects of the system**, from input and output screens to reports, databases, and computer processes.
- That part of the design process that is independent of any specific hardware or software platform is referred to as **logical design**.
 - Theoretically, the **system could be implemented on any hardware and system software**. The idea is to make sure that the system functions as intended.
 - Logical design concentrates on the business aspects of the system and tends to be oriented to a high level of specificity.

Developing Information Systems and the Systems Development Life Cycle (11/16)

- In a **traditional structured approach**, once the **overall high-level design of the system is worked out**, the analysts **begin turning logical specifications into physical ones**. This process is referred to as **physical design**.
 - As part of physical design, analysts **design the various parts of the system to perform the physical operations necessary to facilitate data capture, processing, and information output**.
 - This can be done in many ways, from **creating a working model of the system to be implemented to writing detailed specifications describing all the different parts of the system and how they should be built**.
 - In many cases, the working model becomes the basis for the actual system to be used.
- During physical design, the analyst team **must determine many of the physical details necessary to build the final system**, from the **programming language the system will be written in to the database system that will store the data to the hardware platform on which the system will run**.
 - Often the **choices of language, database, and platform are already decided by the organization or by the client**, and at this point these information technologies must be taken into account in the physical design of the system.
 - In a **structured approach**, the **final product of the design phase is the physical system specifications in a form ready to be turned over to programmers and other system builders for construction**.
 - In an **agile approach**, **logical and physical design become part of the same iterative process and detailed specifications are replaced with multiple working releases of the software**.

Developing Information Systems and the Systems Development Life Cycle (12/16)

- The **fourth phase** in the SDLC is **implementation**. In a structured process, the **physical system specifications, whether in the form of a detailed model or as detailed written specifications, are turned over to programmers** as the first part of the implementation phase.
 - During implementation, analysts **turn system specifications into a working system that is tested and then put into use**.
 - Implementation **includes coding, testing, and installation**.
 - During **coding**, programmers **write the programs that make up the system**.
 - During **testing**, programmers and analysts **test individual programs and the entire system** in order to **find and correct errors**. Following an **agile approach**, **programs are tested as soon as they are written, leading to functional software in a short period of time**.
 - During **installation**, the new system becomes part of the daily activities of the organization. Application software is installed or loaded on existing or new hardware, and users are introduced to the new system and trained.
- **Testing and installation should be planned for as early as the project initiation and planning phase**; both testing and installation require extensive analysis in order to develop exactly the right approach.

Developing Information Systems and the Systems Development Life Cycle (13/16)

- **Implementation activities** also include initial user support such as the **finalization of documentation, training programs, and ongoing user assistance**.
 - Note that documentation and training programs are finalized during implementation; **documentation is produced throughout the life cycle**, and **training (and education) occurs from the inception of a project**.
- **Implementation can continue for as long as the system exists** because ongoing user support is also part of implementation.
 - Despite the best efforts of analysts, managers, and programmers, however, installation is not always a simple process.
 - Many well-designed systems have failed because the installation process was faulty. Even a well-designed system can fail if implementation is not well managed.

Developing Information Systems and the Systems Development Life Cycle (14/16)

- The **fifth and final phase** in the SDLC is **maintenance**. When a system (including its training, documentation, and support) is operating in an organization, **users sometimes find problems with how it works and often think of better ways to perform its functions**. Also, the organization's **needs with respect to the system change over time**.
 - In maintenance, programmers make the **changes that users ask for and modify the system to reflect evolving business conditions**. These changes are necessary to keep the system running and useful.
 - In a sense, **maintenance is not a separate phase but a repetition of the other life-cycle phases required to study and implement the needed changes**.
 - One might think of **maintenance as an overlay on the life cycle rather than as a separate phase**.
 - The amount of **time and effort devoted to maintenance depends greatly on the performance of the previous phases of the life cycle**.
 - There **inevitably comes a time**, however, when an **information system is no longer performing as desired, maintenance costs become prohibitive**, or an organization's needs have changed substantially. Such problems **indicate that it is time to begin designing the system's replacement**, thereby completing the loop and starting the life cycle over again.
 - Often the **distinction between major maintenance and new development is not clear**, which is another reason maintenance often resembles the life cycle itself.

Developing Information Systems and the Systems Development Life Cycle (15/16)

- The SDLC is a **highly linked set of phases whose products feed the activities in subsequent phases.**
- Throughout the SDLC, the **systems development project itself must be carefully planned and managed.**
 - The **larger the systems project, the greater the need for project management.**
 - Several project management techniques have been developed over the past decades, and many have been made more useful through automation.

Developing Information Systems and the Systems Development Life Cycle (16/16)

Products of SDLC Phases	
Phase	Products, Outputs, or Deliverables
Planning	Priorities for systems and projects; an architecture for data, networks, and selection hardware, and information systems management are the result of associated systems
	Detailed steps, or work plan, for project
	Specification of system scope and planning and high-level system requirements or features
	Assignment of team members and other resources
	System justification or business case
Analysis	Description of current system and where problems or opportunities exist, with a general recommendation on how to fix, enhance, or replace current system
Design	Explanation of alternative systems and justification for chosen alternative
	Functional, detailed specifications of all system elements (data, processes, inputs, and outputs)
	Technical, detailed specifications of all system elements (programs, files, network, system software, etc.)
	Acquisition plan for new technology
Implementation	Code, documentation, training procedures, and support capabilities
Maintenance	New versions or releases of software with associated updates to documentation, training, and support

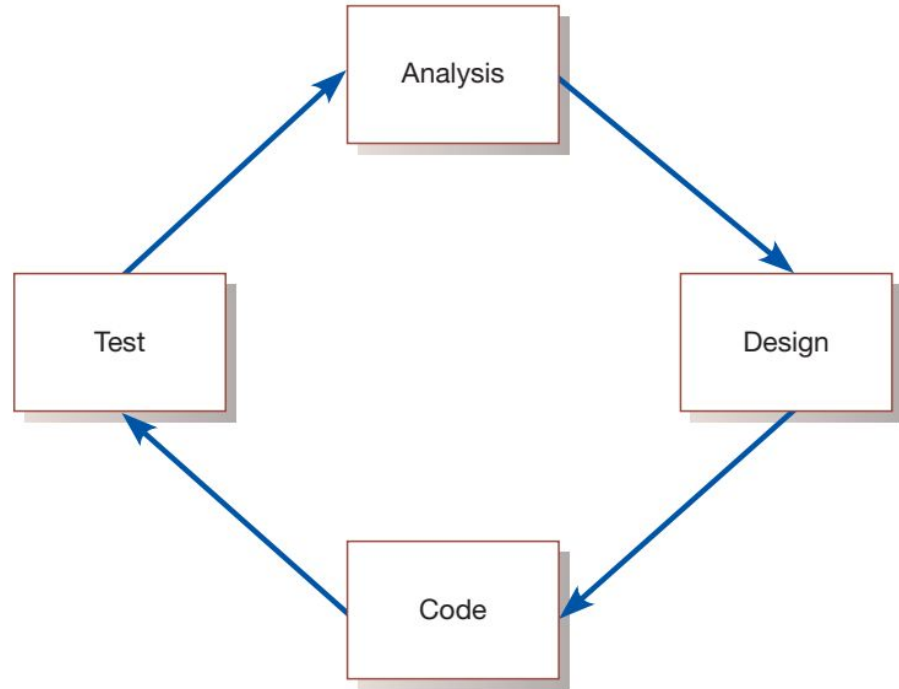
The Heart of the Systems Development Process (1/3)

- **The SDLC provides a convenient way to think about the processes involved in system development.** The different phases are clearly defined, their relationships to one another are well specified, and the sequencing of phases from one to the next, from beginning to end, has a compelling logic.
- In many ways, though, the **SDLC is fiction**.
 - Although almost all systems development projects adhere to some type of life cycle, the **exact location of activities and the specific sequencing of steps can vary greatly from one project to the next**.
- **Current practice combines the activities traditionally thought of as belonging to analysis, design, and implementation into a single process.**
 - Instead of systems requirements being produced in analysis, systems specifications being created in design, and coding and testing being done at the beginning of implementation, current practice combines all of these activities into a single **analysis-design-code-test process**.
 - These activities are the heart of system development. This combination of activities is typical of current practices in **agile methodologies**.
 - Two well-known instances of agile methodologies are **eXtreme Programming (XP)** and **Scrum**, although there are other variations.

The Heart of the Systems Development Process (2/3)

FIGURE 1-6

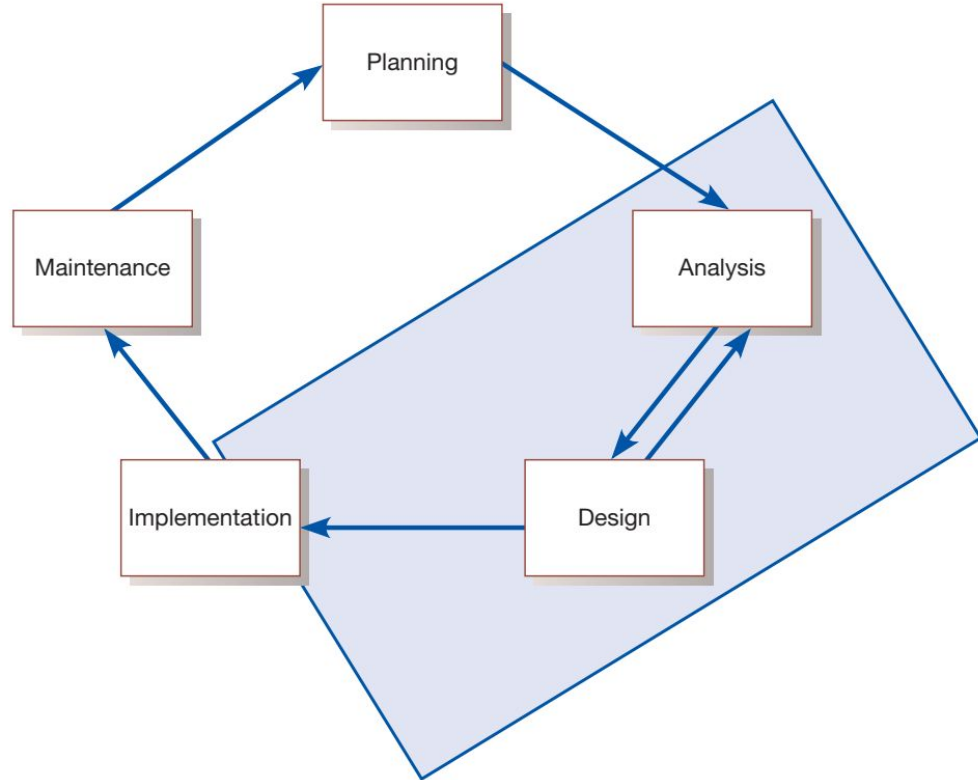
Analysis–design–code–test loop



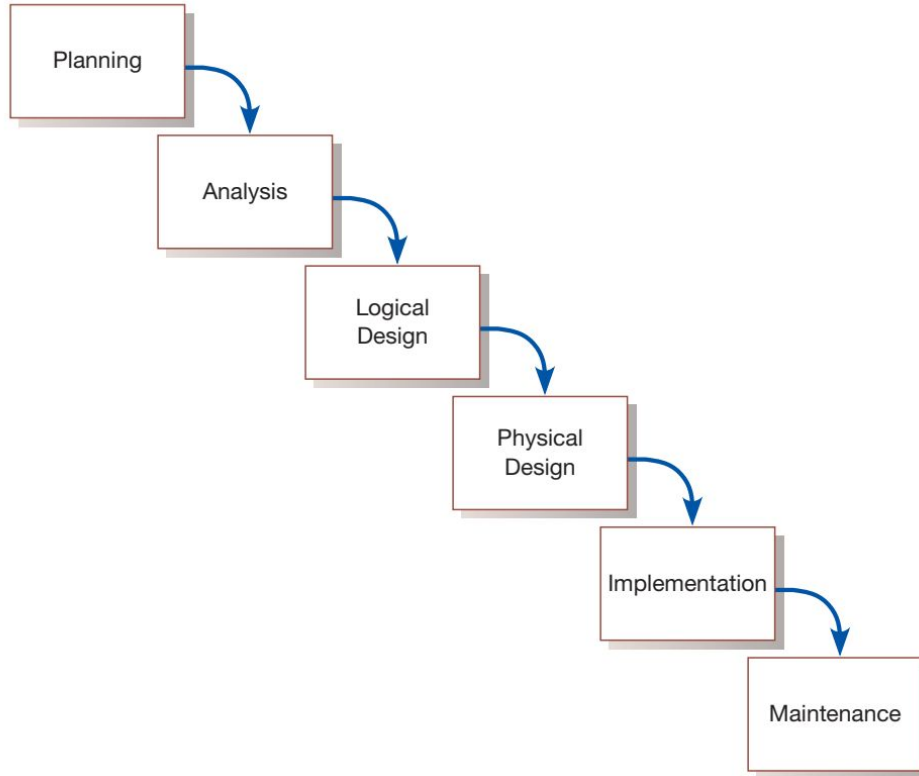
The Heart of the Systems Development Process (3/3)

FIGURE 1-7

Heart of systems development



Traditional Waterfall SDLC (1/5)



Traditional Waterfall SDLC (2/5)

- There are several criticisms of the traditional life-cycle approach to systems development; one relates to the way the life cycle is organized. To better understand these criticisms, it is best to see the form in which the life cycle has traditionally been portrayed, the so-called **waterfall**.
 - Note **how the flow of the project begins in the planning phase and from there runs “downhill” to each subsequent phase, just like a stream that runs off a cliff.**
 - Although the original developer of the waterfall model **called for feedback between phases in the waterfall**, this **feedback came to be ignored** in implementation.
 - It became **too tempting to ignore the need for feedback and to treat each phase as complete unto itself**, never to be revisited once finished.

Traditional Waterfall SDLC (3/5)

- Traditionally, **one phase ended and another began** once a milestone had been reached. The **milestone usually took the form of some deliverable or prespecified output from the phase.**
 - For example, the design deliverable is the set of detailed physical design specifications.
 - Once the milestone had been reached and the new phase initiated, it became difficult to go back. Even though business conditions continued to change during the development process and analysts were pressured by users and others to alter the design to match changing conditions, it was necessary for the analysts to freeze the design at a particular point and go forward.
 - The enormous amount of effort and time necessary to implement a specific design meant that **it would be very expensive to make changes in a system once it was developed.**
 - The traditional waterfall life cycle, then, had the property of **locking users into requirements that had been previously determined, even though those requirements might have changed.**

Traditional Waterfall SDLC (4/5)

- Yet another criticism of the traditional waterfall SDLC is that the **role of system users or customers was narrowly defined.**
 - User roles were **often relegated to the requirements determination or analysis phases of the project**, where it was assumed that all of the requirements could be specified in advance.
 - Such an assumption, coupled with **limited user involvement, reinforced the tendency of the waterfall model to lock in requirements too early, even after business conditions had changed.**

Traditional Waterfall SDLC (5/5)

- In addition, under the traditional waterfall approach, **nebulous and intangible processes such as analysis and design are given hard-and-fast dates for completion, and success is overwhelmingly measured by whether those dates are met.**
 - The focus on milestone deadlines instead of on obtaining and interpreting feedback from the development process leads to **too little focus on doing good analysis and design.**
 - The focus on deadlines **leads to systems that do not match users' needs and that require extensive maintenance, unnecessarily increasing development costs.**
 - **Finding and fixing a software problem after the delivery of the system is often far more expensive than finding and fixing it during analysis and design.**
 - **The result of focusing on deadlines rather than on good practice is unnecessary rework and maintenance effort, both of which are expensive.**
 - According to some estimates, **maintenance costs account for 40 to 70 percent of system development costs.**
- Given these problems, people working in systems development began to look for better ways to conduct systems analysis and design.

CASE Tools (1/4)

- Other efforts to improve the systems development process have taken advantage of the **benefits offered by computing technology itself**.
 - The result has been the creation and fairly widespread use of **computer-aided software engineering (CASE) tools**.
 - CASE tools have been developed for internal use and for sale by several leading firms, but the best known is the series of Rational tools made by IBM
- CASE tools are used to **support a wide variety of SDLC activities**.
 - CASE tools can be used to help in multiple phases of the SDLC: project identification and selection, project initiation and planning, analysis, design, and implementation and maintenance.
- An integrated and standard database called a **repository** is the common method for providing product and tool integration and has been a key factor in enabling CASE to more easily manage larger, more complex projects and to seamlessly integrate data across various tools and products.
 - The idea of a **central repository** of information about a project is not new—the manual form of such a repository is called a project dictionary or workbook.

CASE Tools (2/4)

- The general types of CASE tools are listed below:
 - **Diagramming tools** enable system processes, data, and control structures to be represented graphically.
 - **Computer display and report generators** help prototype how systems “look and feel.” Display (or form) and report generators make it easier for the systems analyst to identify data requirements and relationships.
 - **Analysis tools** automatically check for incomplete, inconsistent, or incorrect specifications in diagrams, forms, and reports.
 - **A central repository** enables the integrated storage of specifications, diagrams, reports, and project management information.
 - **Documentation generators** produce technical and user documentation in standard formats.
 - **Code generators** enable the automatic generation of program and database definition code directly from the design documents, diagrams, forms, and reports.

CASE Tools (3/4)

Examples of CASE Usage within the SDLC

SDLC Phase	Key Activities	CASE Tool Usage
Project identification and selection	Display and structure high-level organizational information	Diagramming and matrix tools to create and structure information
Project initiation and planning	Develop project scope and feasibility	Repository and documentation generators to develop project plans
Analysis	Determine and structure system requirements	Diagramming to create process, logic, and data models
Logical and physical design	Create new system designs	Form and report generators to prototype designs; analysis and documentation generators to define specifications
Implementation	Translate designs into an information system	Code generators and analysis, form and report generators to develop system; documentation generators to develop system and user documentation
Maintenance	Evolve information system	All tools are used (repeat life cycle)

CASE Tools (4/4)

- **CASE helps programmers and analysts do their jobs more efficiently and more effectively by automating routine tasks.**
 - However, many organizations that use CASE tools do not use them to support all phases of the SDLC. Some organizations may extensively use the diagramming features but not the code generators.
 - There are a variety of reasons why organizations choose to adopt CASE partially or not use it at all.
 - These reasons range from a **lack of vision for applying CASE to all aspects** of the SDLC to the belief that CASE technology will fail to meet an organization's unique system development needs.
 - In some organizations, **CASE has been extremely successful, whereas in others it has not.**

Other Approaches

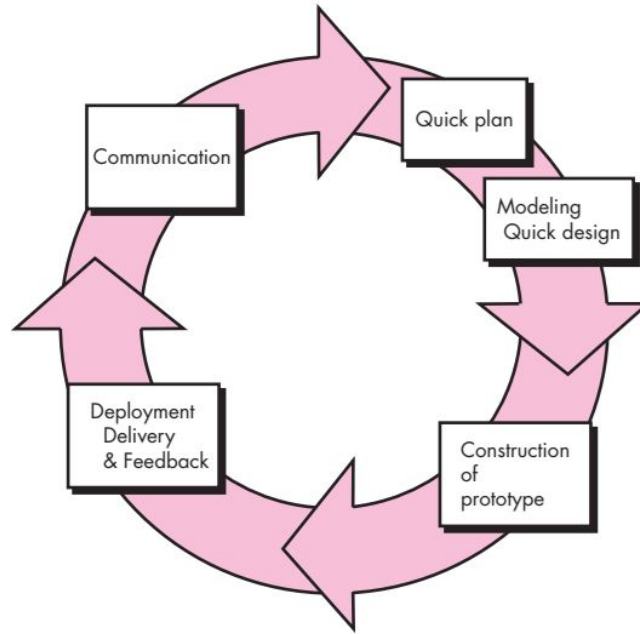
Prototyping (1/6)

- **Software, like all complex systems, evolves over a period of time.** In these and similar situations, we need a process model that has been explicitly designed to accommodate a product that evolves over time.
 - **Business and product requirements often change** as development proceeds, making a straight-line path to an end product unrealistic;
 - **Tight market deadlines** make completion of a comprehensive software product impossible, but a limited version must be introduced to meet competitive or business pressure.
 - A set of core product or system requirements is well understood, but **the details of product or system extensions have yet to be defined.**
- Evolutionary models are iterative. They are characterized in a manner that enables you to develop increasingly more complete versions of the software.

Prototyping (2/6)

- In these and many other situations, a prototyping paradigm may offer the best approach.
 - Often, **a customer defines a set of general objectives for software** but does not identify detailed requirements for functions and features.
 - In other cases, the **developer may be unsure of the efficiency of an algorithm**, the adaptability of an operating system, or the form that human-machine interaction should take.
- Although prototyping can be used as a stand-alone process model, it is **more commonly used as a technique that can be implemented within the context of any one of the process models**. Regardless of the manner in which it is applied, the prototyping paradigm assists you and other stakeholders to better understand what is to be built when requirements are fuzzy.

Prototyping (3/6)



Prototyping (4/6)

- The prototyping paradigm **begins with communication**. We meet with other stakeholders to define the overall objectives for the software, identify whatever requirements are known, and outline areas where further definition is mandatory.
- A prototyping **iteration is planned quickly, and modeling (in the form of a “quick design”) occurs**.
 - A **quick design focuses on a representation of those aspects of the software that will be visible to end users** (e.g., human interface layout or output display formats). The quick design leads to the construction of a prototype.
 - The prototype is **deployed and evaluated by stakeholders, who provide feedback that is used to further refine requirements**.
 - **Iteration occurs as the prototype is tuned to satisfy the needs of various stakeholders** while at the same time enabling you to better understand what needs to be done.
- Ideally, the prototype **serves as a mechanism for identifying software requirements**. If a working prototype is to be built, you can make use of an existing program. fragments or apply tools (e.g., report generators and window managers) that enable working programs to be generated quickly.

Prototyping (5/6)

- But what do you do with the prototype when it has served the purpose described earlier?
 - The prototype can serve as “**the first system.**” In an idealized view, it is **thrown away**. Although some prototypes are built as “throwaways,” others are evolutionary in the sense that the **prototype slowly evolves into the actual system**.
- Both stakeholders and software engineers like the prototyping paradigm. Users get a feel for the actual system, and developers get to build something immediately.

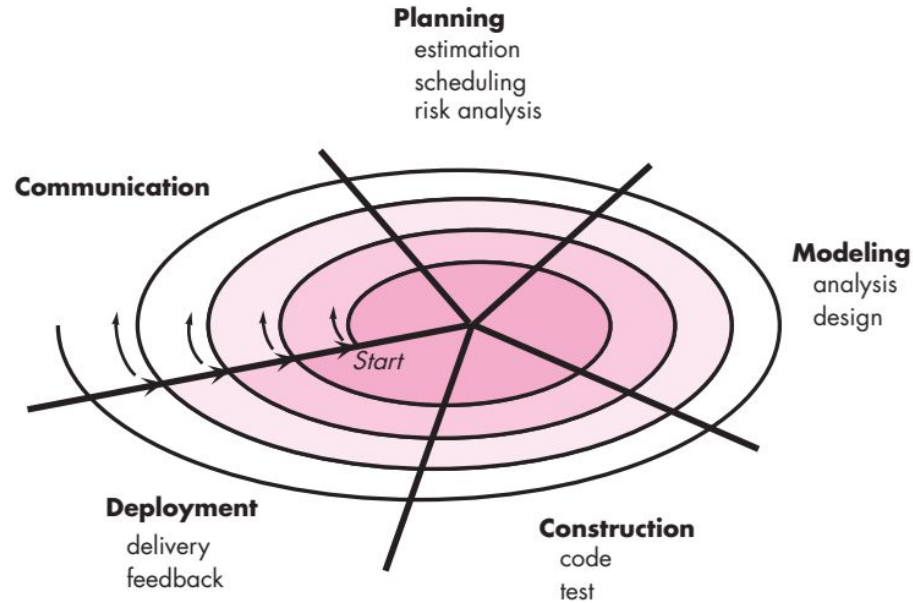
Prototyping (6/6)

- Prototyping can be problematic for the following reasons:
 - Stakeholders see what appears to be a working version of the software, **unaware that the prototype is held together haphazardly, unaware that in the rush to get it working you haven't considered overall software quality or long-term maintainability.** When informed that the product must be rebuilt so that high levels of quality can be maintained, stakeholders cry foul and demand that "a few fixes" be applied to make the prototype a working product. Too often, software development management relents.
 - As a software engineer, you often make implementation compromises in order to get a prototype working quickly. An inappropriate operating system or programming language may be used simply because it is available and known; an inefficient algorithm may be implemented simply to demonstrate capability. **After a time, you may become comfortable with these choices and forget all the reasons why they were inappropriate.** The less-than-ideal choice has now become an integral part of the system.
- Although problems can occur, **prototyping can be an effective paradigm for software engineering.** The key is to define the rules of the game at the beginning; that is, all stakeholders should agree that the prototype is built to serve as a mechanism for defining requirements. It is then discarded (at least in part), and the actual software is engineered with an eye toward quality.

Spiral Model (1/5)

- The spiral model is an evolutionary software process model that couples the **iterative nature of prototyping with the controlled and systematic aspects of the waterfall model**. It provides the potential for **rapid development of increasingly more complete versions of the software**.
- The spiral development model is a **risk-driven process** model generator that is **used to guide multi-stakeholder concurrent engineering of software-intensive systems**.
- It has two main distinguishing features.
 - One is a **cyclic approach** for incrementally growing a system's degree of definition and implementation while decreasing its degree of risk.
 - The other is **a set of anchor point milestones** for ensuring stakeholder commitment to feasible and mutually satisfactory system solutions.

Spiral Model (2/5)



Spiral Model (3/5)

- Using the spiral model, software is developed in a series of evolutionary releases.
 - During early iterations, **the release might be a model or prototype.**
 - During later iterations, **increasingly more complete versions of the engineered system are produced.**
- The **first circuit** around the spiral might result in the development of a **product specification; subsequent passes** around the spiral might be used to **develop a prototype** and **then progressively more sophisticated versions of the software.** Each pass through the **planning region results in adjustments to the project plan.** Cost and schedule are adjusted based on feedback derived from the customer after delivery. In addition, the project manager adjusts the planned number of iterations required to complete the software.

Spiral Model (4/5)

- Unlike other process models that end when software is delivered, **the spiral model can be adapted to apply throughout the life of the computer software.**
- Therefore, the first circuit around the spiral might represent a “**concept development project**” that starts at the core of the spiral and continues for multiple iterations until concept development is complete. If the concept is to be developed into an actual product, the process proceeds outward on the spiral, and a “**new product development project**” commences. The new product will evolve through a number of iterations around the spiral. Later, a circuit around the spiral might be used to represent a “**product enhancement project.**”
- In essence, the spiral, when characterized in this way, **remains operative until the software is retired.** There are times when the **process is dormant**, but whenever a change is initiated, the process starts at the appropriate entry point (e.g., product enhancement).

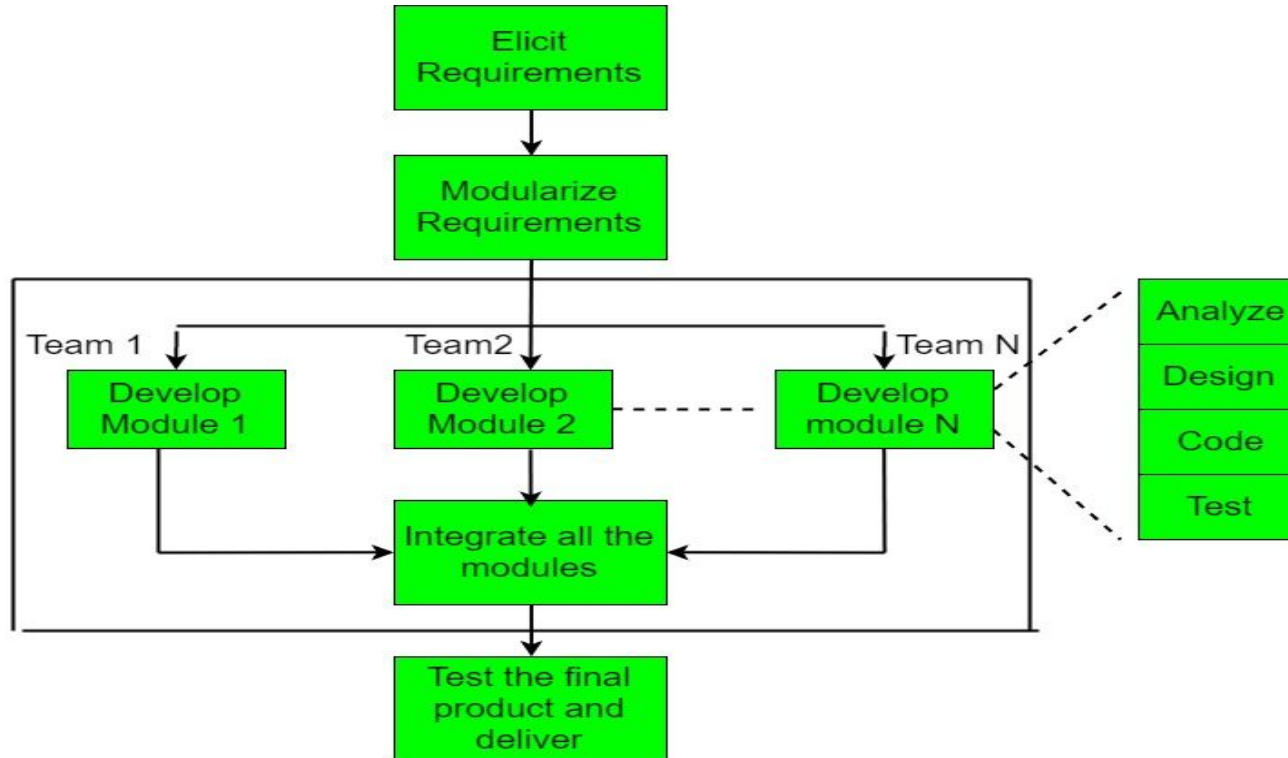
Spiral Model (5/5)

- The spiral model is **a realistic approach to the development of large-scale systems and software.**
 - Because software evolves as the process progresses, the developer and customer better understand and react to risks at each evolutionary level.
 - The spiral model uses prototyping as a risk reduction mechanism but, more importantly, enables you to apply the prototyping approach at any stage in the evolution of the product.
 - It maintains the systematic stepwise approach suggested by the classic life cycle but incorporates it into an iterative framework that more realistically reflects the real world.
 - The spiral model demands a direct consideration of technical risks at all stages of the project and, if properly applied, should reduce risks before they become problematic.
- But like other paradigms, **the spiral model is not a panacea.** It may be **difficult to convince customers** (particularly in contract situations) that the evolutionary approach is controllable. It **demand considerable risk assessment expertise and relies on this expertise for success.** If a major risk is not uncovered and managed, problems will undoubtedly occur.

Rapid Application Development (1/4)

- Rapid application development (RAD) is an **agile software development** approach that **focuses more on ongoing software projects and user feedback and less on following a strict plan**. As such, it **emphasizes rapid prototyping over costly planning**.
- Though often mistaken for a specific model, **RAD is the idea that we benefit by treating our software projects like clay rather than steel**, in the sense that they can be tested and refined instead of following detailed planning and design phases that are typically associated with traditional development models.
- The critical feature of this model is **the use of powerful development tools and techniques**. A software project can be implemented using this model **if the project can be broken down into small modules wherein each module can be assigned independently to separate teams**. These modules can finally be combined to form the final product. Development of each module involves the various basic steps as in the waterfall model, i.e., analyzing, designing, coding, and then testing, etc. Another striking feature of this model is a short period, i.e., **the time frame for delivery is generally 60-90 days**.

Rapid Application Development (2/4)



Rapid Application Development (3/4)

- This model consists of 4 basic phases:
 - **Requirements Planning:** This involves *the use of various techniques used in requirements elicitation, like brainstorming, task analysis, form analysis, user scenarios, FAST (Facilitated Application Development Technique), etc.* It also consists of the entire structured plan describing the critical data, methods to obtain it, and then processing it to form a final refined model.
 - **User Description:** This phase consists of *taking user feedback and building the prototype using developer tools.* In other words, it includes re-examination and validation of the data collected in the first phase. The dataset attributes are also identified and elucidated in this phase.
 - **Construction:** In this phase, *refinement of the prototype and delivery take place.* It includes the actual use of powerful automated tools to transform processes and data models into the final working product. All the required modifications and enhancements are to be done in this phase.
 - **Cutover:** *All the interfaces between the independent modules developed by separate teams have to be tested properly.* The use of powerfully automated tools and subparts makes testing easier. This is followed by acceptance testing by the user.

Rapid Application Development (4/4)

- When to use the RAD Model?
 - **Well-understood requirements:** When project requirements are stable and transparent, RAD is appropriate.
 - **Time-sensitive Projects:** This is suitable for projects that need to be developed and delivered quickly due to tight deadlines.
 - **Small to Medium-Sized Projects:** This model is better suited for smaller initiatives requiring a controllable number of team members.
 - **High User Involvement:** This model fits where ongoing input and interaction from users are essential.
 - **Innovation and Creativity:** This model is helpful for tasks requiring creative inquiry and innovation.
 - **Prototyping:** It is necessary when developing and improving prototypes and is a key component of the development process.
 - **Low technological complexity:** This model is suitable for tasks using comparatively straightforward technological specifications.

Agile Methodologies (1/7)

- According to Fowler (2003), the Agile methodologies share three key principles:
 - A focus on adaptive rather than predictive methodologies.
 - A focus on people rather than roles.
 - A focus on self-adaptive processes.
- The Agile Methodologies group argues that **software development methodologies adapted from engineering generally do not fit with real-world software development.**
 - In engineering disciplines, such as civil engineering, **requirements tend to be well understood.** For software, on the other hand, **requirements are rarely well understood, and they change continually during the lifetime of the project.**

Agile Methodologies (2/7)

- One mechanism for dealing with **a lack of predictability**, which all Agile methodologies share, is **iterative development**.
 - Iterative development **focuses on the frequent production of working versions of a system** that have a subset of the total number of required features.
 - Iterative development **provides feedback to customers and developers alike**.
- The Agile Methodologies' **focus on people** is an emphasis on individuals rather than on the roles that people perform. The roles that people fill, such as systems analyst, tester or manager, are not as important as the individuals who fill those roles.
- Fowler argues that the application of engineering principles to systems development has resulted in a **view of people as interchangeable units** instead of a view of people as talented individuals, each bringing something unique to the development team

Agile Methodologies (3/7)

The Agile Manifesto

The Manifesto for Agile Software Development

Seventeen anarchists agree:

We are uncovering better ways of developing software by doing it and helping others do it.

Through this work we have come to value:

- *Individuals and interactions* over processes and tools.
- *Working software* over comprehensive documentation.
- *Customer collaboration* over contract negotiation.
- *Responding to change* over following a plan.

That is, while we value the items on the right, we value the items on the left more. We follow the following principles:

- Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
- Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
- Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
- Businesspeople and developers work together daily throughout the project.
- Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
- The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
- Working software is the primary measure of progress.
- Continuous attention to technical excellence and good design enhances agility.
- Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- Simplicity—the art of maximizing the amount of work not done—is essential.
- The best architectures, requirements, and designs emerge from self-organizing teams.
- At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

—Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, Dave Thomas (www.agileAlliance.org)

Agile Methodologies (4/7)

- The Agile methodologies **promote a self-adaptive software development process**.
 - As software is developed, **the process used to develop it should be refined and improved**. Development teams can do this through a **review process, often associated with the completion of iterations**.
 - The **implication** is that, as processes are adapted, one would **not expect to find a single monolithic methodology** within a given corporation or enterprise. Instead, one would **find many variations of the methodology**, each of which reflects the particular talents and experience of the team using it.
- Agile methodologies **are not for every project**. Fowler (2003) recommends an agile or adaptive process if your project involves
 - unpredictable or dynamic requirements,
 - responsible and motivated developers, and
 - customers who understand the process and will get involved.

Agile Methodologies (5/7)

- A **more engineering-oriented, predictable process** may be called for if the development team exceeds 100 people or if the project is **operating under a fixed-price or fixed-scope contract**.
- In fact, whether a systems development project is **organized in terms of Agile or more traditional methodologies depends on many different considerations**.
 - If a project is **considered high-risk and highly complex** and has a **development team made up of hundreds of people**, then more **traditional methods will apply**. **Less risky, smaller, and simpler development efforts** lend themselves more to **Agile methods**.
 - Other determining factors include **organizational practice and standards** and the extent to which **different parts of the system will be contracted out to others for development**. Obviously, the **larger the proportion of the system that will be outsourced, the more detailed the design specifications will need to be so that subcontractors can understand what is needed**.

Agile Methodologies (6/7)



Five Critical Factors That Distinguish Agile and Traditional Approaches to Systems Development

Factor	Agile Methods	Traditional Methods
Size	Well matched to small products and teams. Reliance on tacit knowledge limits scalability.	Methods evolved to handle large products and teams. Hard to tailor down to small projects.
Criticality	Untested on safety-critical products. Potential difficulties with simple design and lack of documentation.	Methods evolved to handle highly critical products. Hard to tailor down to products that are not critical.
Dynamism	Simple design and continuous refactoring are excellent for highly dynamic environments but a source of potentially expensive rework for highly stable environments.	Detailed plans and Big Design Up Front, excellent for highly stable environment but a source of expensive rework for highly dynamic environments.
Personnel	Requires continuous presence of a critical mass of scarce experts. Risky to use non-agile people.	Needs a critical mass of scarce experts during project definition but can work with fewer later in the project, unless the environment is highly dynamic.
Culture	Thrives in a culture where people feel comfortable and empowered by having many degrees of freedom (thriving on chaos).	Thrives in a culture where people feel comfortable and empowered by having their roles defined by clear practices and procedures (thriving on order).

Agile Methodologies (7/7)

- Many different individual methodologies come under the umbrella of Agile methodologies.
 - Crystal family of methodologies,
 - Adaptive Software Development,
 - Scrum,
 - Feature-driven development,
 - eXtreme Programming
 - and others.

eXtreme Programming (1/3)

- eXtreme programming (XP) is an approach to software development distinguished by its
 - short cycles,
 - incremental planning approach,
 - focus on automated tests written by programmers and customers to monitor the development process, and
 - reliance on an evolutionary approach to development that lasts throughout the lifetime of the system.
- Key emphases of eXtreme programming are its
 - use of two-person programming teams (pair programming) and
 - having a customer on-site during the development process.

eXtreme Programming (2/3)

- The relevant parts of eXtreme Programming that relate to **design specifications** are
 - **how planning, analysis, design, and construction are all fused into a single phase of activity;** and
 - **its unique way of capturing and presenting system requirements and design specifications.**
- With XP, **all phases of the life cycle converge into a series of activities** based on the basic processes of coding, testing, listening, and designing.
- Under this approach, **coding and testing are intimately related parts** of the same process.
 - The **programmers who write the code also develop the tests.**
 - The emphasis is on **testing those things that can break or go wrong, not on testing everything.**
 - Code is tested very soon after it is written.
- The overall philosophy behind XP is that the **code will be integrated into the system it is being developed for and tested within a few hours after it has been written.**
 - If all the tests run successfully, then development proceeds. If not, the code is reworked until the tests are successful.

eXtreme Programming (3/3)

- Another part of XP that makes the code-and-test process work more smoothly is the practice of **pair programming**.
 - All **coding and testing is done by two people working together** to write code and develop tests.
 - Pair programming is not one person typing while the other one watches; rather, the **two programmers work together on the problem they are trying to solve, exchanging information and insight and sharing skills.**
- Compared to traditional coding practices, the **advantages of pair programming** include:
 - more (and better) communication among developers,
 - higher levels of productivity,
 - higher-quality code, and
 - reinforcement of the other practices in XP, such as the code and-test discipline.
- Although the XP process has its advantages, just as with any other approach to systems development, **it is not for everyone and is not applicable to every project.**

Managing the Information Systems Project

Introduction (1/2)

- Throughout the SDLC, the **project manager** is responsible for **initiating, planning, executing, and closing down the systems development project**.
 - Project management is arguably the most important aspect of an information systems development project. **Effective project management helps to ensure that systems development projects meet customer expectations and are delivered within budget and time constraints.**
- Today, there is a **shift in the types of projects most firms are undertaking**, which makes project management much more difficult and even more critical to project success.
 - For example, in the past, **organizations focused much of their development on very large, custom-designed, stand-alone applications.**
 - Today, **much of the systems development effort in organizations focuses on implementing packaged software** such as enterprise resource planning (ERP) and data warehousing systems.

Introduction (2/2)

- **Existing legacy applications are also being modified** so that business-to-business transactions can occur seamlessly over the Internet.
 - **New web-based interfaces are being added to existing legacy systems** so that a broader range of users, often **distributed globally**, can access corporate information and systems.
- Additionally, **software developed by global outsourcing partners that must be integrated into an organization's existing portfolio of applications is now common practice.**
 - Working with **vendors to supply applications**, with customers or suppliers to integrate systems, or with a broader and more diverse user community requires that project managers be highly skilled. Consequently, it is important that you **gain an understanding of the project management process**; this will become a **critical skill for your future success**.

Managing the Information Systems Project (1/6)

- **Project management** is an important aspect of the development of information systems and a critical skill for a systems analyst.
 - The **focus of project management is to ensure that systems development projects meet customer expectations and are delivered within budget and time constraints.**
- The **project manager is a systems analyst** with a diverse set of skills—management, leadership, technical, conflict management, and customer relationship—who is responsible for initiating, planning, executing, and closing down a project.
- As a project manager, your environment is **one of continual change and problem solving.**
 - In some organizations, the **project manager is a very experienced systems analyst**, whereas in others, both **junior and senior analysts are expected to take on this role**, managing parts of a project or actively supporting a more senior colleague who assumes the project manager role.
- Understanding the project management process is a critical skill for your future success.

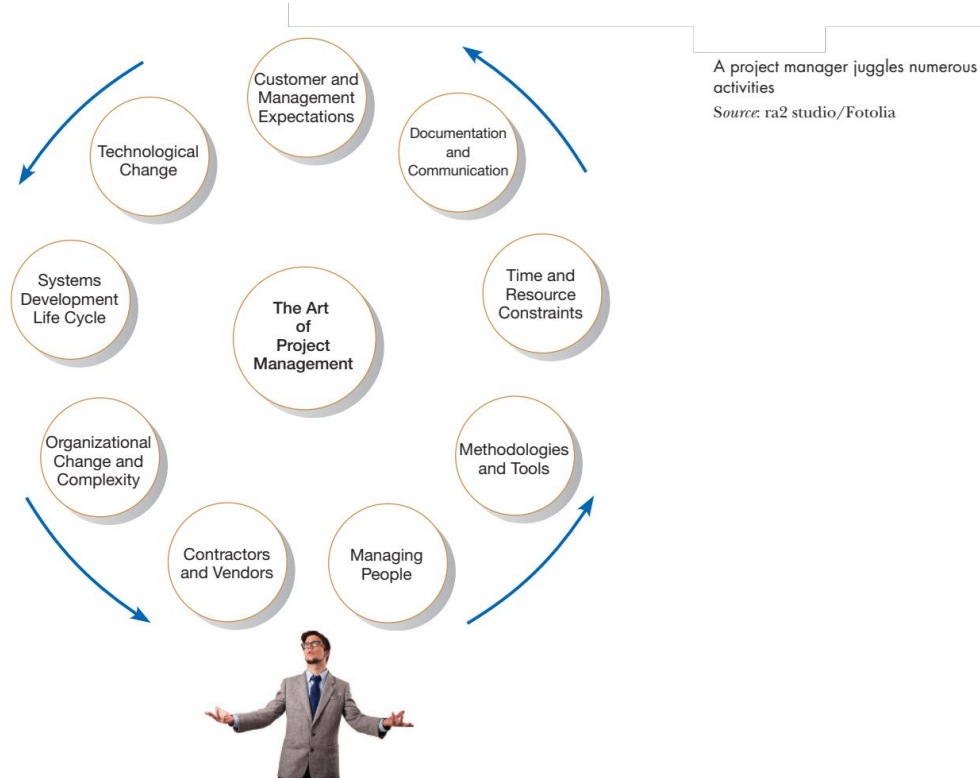
Managing the Information Systems Project (2/6)

- **Creating and implementing successful projects requires managing the resources, activities, and tasks needed to complete the information systems project.**
- A project is a **planned undertaking of a series of related activities** to reach an objective that has a beginning and an end.
 - The first question you might ask yourself is, “**Where do projects come from?**” and, after considering all the different things that you could be asked to work on within an organization, “**How do I know which projects to work on?**” The ways in which each organization answers these questions vary.

Managing the Information Systems Project (3/6)

- **Once a potential project has been identified, an organization must determine the resources required for its completion.**
 - This is done by **analyzing the scope of the project and determining the probability of successful completion.**
 - After getting this information, the organization can then **determine whether taking advantage of an opportunity or solving a particular problem is feasible within time and resource constraints.** A **feasibility study**, which is conducted by the project manager, involves determining if the information system makes sense for the organization from an economic and operational standpoint.
 - If deemed feasible, a **more detailed project analysis is then conducted.**
- As you will see, the ability to determine the size, scope, and resource requirements of a project is just one of the many skills that a project manager must possess. A **project manager is often thought of as a juggler keeping aloft many balls,** which reflect the various aspects of a project's development.

Managing the Information Systems Project (4/6)



Managing the Information Systems Project (5/6)

- To successfully orchestrate the construction of a complex information system, a project manager must have **interpersonal, leadership, and technical skills**.
 - Note that many of the **skills are related to personnel or general management**, not simply technical skills.
- The project management process, which involves **four phases**:
 - Initiating the project
 - Planning the project
 - Executing the project
 - Closing down the project
- **Several activities must be performed during each of these four phases.** Following this formal project management process **greatly increases the likelihood of project success.**

Managing the Information Systems Project (6/6)

Common Activities and Skills of a Project Manager

Activity	Description	Skill
Leadership	Influencing the activities of others toward the attainment of a common goal through the use of intelligence, personality, and abilities	Communication; liaison between management, users, and developers; assigning activities; monitoring progress
Management	Getting projects completed through the effective utilization of resources	Defining and sequencing activities; communicating expectations; assigning resources to activities; monitoring outcomes
Customer relations	Working closely with customers to ensure that project deliverables meet expectations	Interpreting system requests and specifications; site preparation and user training; contact point for customers
Technical problem solving	Designing and sequencing activities to attain project goals	Interpreting system requests and specifications; defining activities and their sequence; making trade-offs between alternative solutions; designing solutions to problems
Conflict management	Managing conflict within a project team to assure that conflict is not too high or too low	Problem solving; smoothing out personality differences; compromising; goal setting
Team management	Managing the project team for effective team performance	Communication within and between teams; peer evaluations; conflict resolution; team building; self-management
Risk and change management	Identifying, assessing, and managing the risks and day-to-day changes that occur during a project	Environmental scanning; risk and opportunity identification and assessment; forecasting; resource redeployment

Managing the Information Systems Project: Initiating a Project (1/6)

- During project initiation, the **project manager performs several activities to assess the size, scope, and complexity of the project and to establish procedures to support subsequent activities.**
 - Depending on the project, **some initiation activities may be unnecessary and some may be very involved.**

Project Initiation

1. Establishing the Project Initiation Team
2. Establishing a Relationship with the Customer
3. Establishing the Project Initiation Plan
4. Establishing Management Procedures
5. Establishing the Project Management Environment and Project Workbook
6. Developing the Project Charter

Managing the Information Systems Project: Initiating a Project (2/6)

- **Establishing the project initiation team:** This activity involves organizing an initial core of project team members to assist in accomplishing the project initiation activities.
- **Establishing a relationship with the customer:** A thorough understanding of your customer builds stronger partnerships and higher levels of trust.
- **Establishing the project initiation plan:** This step defines the activities required to organize the initiation team while it is working to define the goals and scope of the project.
- **Establishing management procedures:** Successful projects require the development of effective management procedures.

Managing the Information Systems Project: Initiating a Project (3/6)

- **Establishing the project management environment and project workbook:** The focus of this activity is to collect and organize the tools that you will use while managing the project and to construct the project workbook.
 - Diagrams, charts, and system descriptions provide much of the project workbook contents. Thus, the project workbook serves as a repository for all project correspondence, inputs, outputs, deliverables, procedures, and standards established by the project team.
 - **Project workbook:** An online or hard-copy repository for all project correspondence, inputs, outputs, deliverables, procedures, and standards that is used for performing project audits, orienting new team members, communicating with management and customers, identifying future projects, and performing post-project reviews.
 - The **project workbook can be stored as an online electronic document or in a large three-ring binder.** The project workbook is used by all team members and is useful for project audits, orientation of new team members, communication with management and customers, identifying future projects, and performing post-project reviews. The establishment of a workbook and the diligent recording of all project information are two of the most important activities you will perform as project manager.

Managing the Information Systems Project: Initiating a Project (4/6)

- **Developing the project charter:** The project charter is a short (typically one page) high-level document prepared for the customer that describes what the project will deliver and outlines many of the key elements of the project.
 - A project charter can vary in the amount of detail it contains, but it often includes the following elements:
 - Project title and date of authorization
 - Project manager name and contact information
 - Customer name and contact information
 - Projected start and completion dates
 - Key stakeholders, project role, and responsibilities
 - Project objectives and description
 - Key assumptions or approach
 - Signature section for key stakeholders
 - The project charter ensures that both you and your customer gain a common understanding of the project. It is also a very useful communication tool; it helps to announce to the organization that a particular project has been chosen for development.

Managing the Information Systems Project: Initiating a Project (5/6)

Pine Valley Furniture		Prepared: November 2, 2017	
Project Charter			
Project Name:		Customer Tracking System	
Project Manager:		Jim Woo (jwoo@pvf.com)	
Customer:		Marketing	
Project Sponsor:		Jackie Judson (jjudson@pvf.com)	
Project Start/End (projected):		10/2/17-2/1/18	
Project Overview:			
This project will implement a customer tracking system for the marketing department. The purpose of this system is to automate the ... to save employee time, reduce errors, have more timely information			
Objectives:			
• Minimize data entry errors • Provide more timely information • ...			
Key Assumptions:			
• System will be built in house • Interface will be a web browser • System will access customer database • ...			
Stakeholders and Responsibilities:			
Stakeholder	Role	Responsibility	Signatures
Jackie Judson	VP Marketing	Project Vision, Resources	<i>Jackie Judson</i>
Alex Datta	CIO	Monitoring, Resources	<i>Alex Datta</i>
Jim Woo	Project Manager	Planning, Monitoring, Executing Project	<i>Jim Woo</i>
James Jordan	Director of Sales	System Functionality	<i>James Jordan</i>
Mary Shide	VP Human Resources	Staff Assignments	<i>Mary Shide</i>

Managing the Information Systems Project: Initiating a Project (6/6)

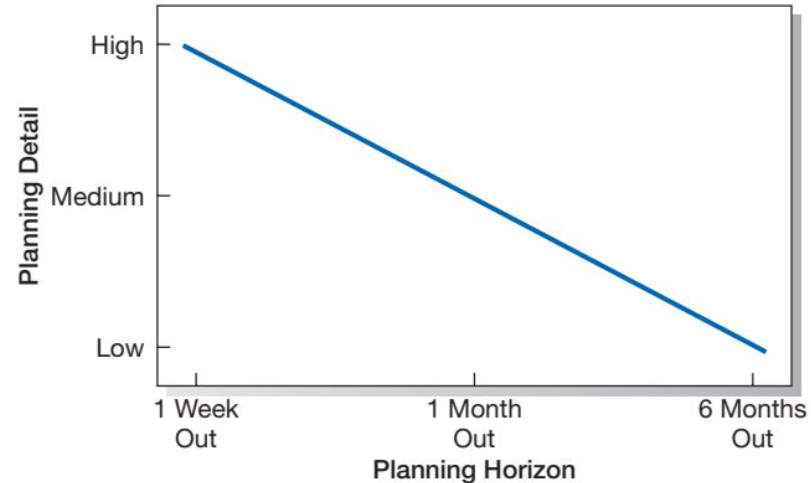
- **Project initiation is complete once these six activities have been performed.**
- **Before moving on to the next phase of the project, the work performed during project initiation is reviewed at a meeting attended by management, customers, and project team members.**
- **An outcome of this meeting is a decision to continue, modify, or abandon the project.**

Managing the Information Systems Project: Planning the Project (1/24)

- The next step in the project management process is **project planning**.
 - Research has found a **positive relationship between effective project planning and positive project outcomes**.
 - Project planning **involves defining clear, discrete activities and the work needed to complete each activity within a single project**.
 - It often **requires you to make numerous assumptions about the availability of resources** such as hardware, software, and personnel.
 - It is **much easier to plan nearer-term activities** than those occurring in the future.
- In actual fact, you often have to **construct longer-term plans that are more general in scope and nearer-term plans that are more detailed**.
- The **repetitive nature of the project management process** requires that **plans be constantly monitored throughout the project and periodically updated** (usually after each phase) based upon the most recent information.

Managing the Information Systems Project: Planning the Project (2/24)

- The principle that **nearer-term plans are typically more specific and firmer than longer-term plans.**
 - For example, it is **virtually impossible to rigorously plan activities late in the project without first completing the earlier activities.**
 - Also, the **outcome of activities performed earlier in the project is likely to affect later activities.**
 - This means that it is **very difficult, and very likely inefficient, to try to plan detailed solutions for activities that will occur far into the future.**



Managing the Information Systems Project: Planning the Project (3/24)

- As with the project initiation process, **varied and numerous activities must be performed during project planning.**
- The types of activities that you can perform during project planning are summarized in the figure.

Project Planning

1. Describing Project Scope, Alternatives, and Feasibility
2. Dividing the Project into Manageable Tasks
3. Estimating Resources and Creating a Resource Plan
4. Developing a Preliminary Schedule
5. Developing a Communication Plan
6. Determining Project Standards and Procedures
7. Identifying and Assessing Risk
8. Creating a Preliminary Budget
9. Developing a Project Scope Statement
10. Setting a Baseline Project Plan

Managing the Information Systems Project: Planning the Project (4/24)

- **Describing project scope, alternatives, and feasibility:** The purpose of this activity is to understand the content and complexity of the project.
 - During this activity, you should reach agreement on the following questions:
 - What problem or opportunity does the project address?
 - What are the quantifiable results to be achieved?
 - What needs to be done?
 - How will success be measured?
 - How will we know when we are finished?
 - After defining the scope of the project, your next objective is to identify and document **general alternative solutions for the current business problem or opportunity**.
 - You must then **assess the feasibility of each alternative solution** and **choose which to consider during subsequent SDLC phases**.
 - In some instances, off-the-shelf software can be found. It is also important that any unique problems, constraints, and assumptions about the project be clearly stated.

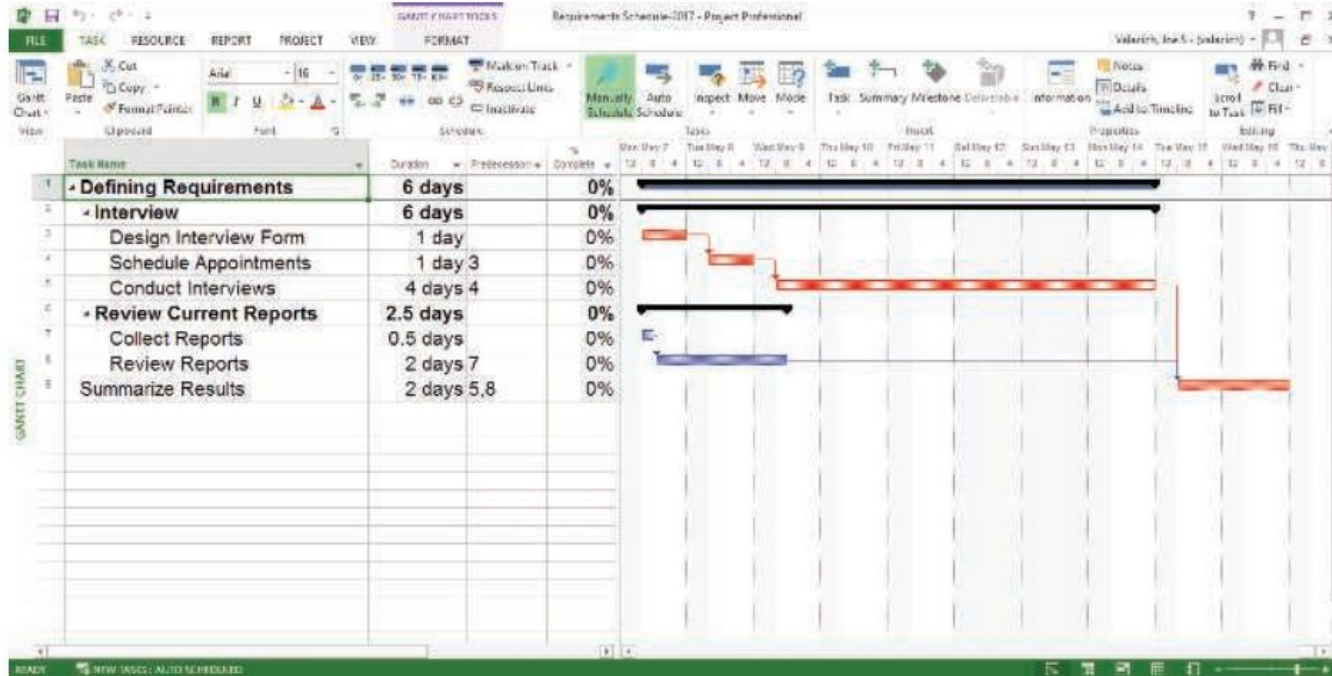
Managing the Information Systems Project: Planning the Project (5/24)

- **Dividing the project into manageable tasks:** This is a critical activity during the project planning process. Here, you must **divide the entire project into manageable tasks and then logically order them to ensure a smooth evolution between tasks**. The definition of tasks and their sequence is referred to as the **work breakdown structure**.
 - Some tasks may be **performed in parallel**, whereas **others must follow one another sequentially**.
 - Task **sequence depends on**
 - **which tasks produce deliverables needed in other tasks,**
 - **when critical resources are available,**
 - **the constraints placed on the project by the client, and**
 - **the process outlined in the SDLC.**

Managing the Information Systems Project: Planning the Project (6/24)

- A work breakdown for the activities can be represented using a **Gantt chart**.
 - A Gantt chart is a graphical representation of a project that shows each task as a horizontal bar whose length is proportional to its time for completion. Different colors, shades, or shapes can be used to highlight each kind of task.
- Most **project management software** tools support a broad range of task durations, including minutes, hours, days, weeks, and months.
- The **SDLC** consists of several phases that you will need to break down into activities.
 - Creating a work breakdown structure requires that you **decompose phases into activities—summary tasks—and activities into specific tasks**.
 - For example, the activity of interviewing consists of three tasks: design an interview form, schedule appointments, and conduct interviews.

Managing the Information Systems Project: Planning the Project (7/24)



Gantt chart showing project tasks, duration times for those tasks, and predecessors

(Source: Microsoft Corporation.)

Managing the Information Systems Project: Planning the Project (8/24)

- **Defining tasks in too much detail will make the management of the project unnecessarily complex.**
 - You will develop the **skill of discovering the optimal level of detail** for representing tasks through experience.
 - For example, it may be very difficult to list tasks **that require less than one hour of time** to complete in a final work breakdown structure.
 - Alternatively, **choosing tasks that are too large in scope** (e.g., several weeks long) will not provide you with a clear sense of the status of the project or of the interdependencies between tasks.
- What are the characteristics of "task"? A task
 - can be done by one person or a well-defined group,
 - has a single and identifiable deliverable (the task is, however, the process of
 - creating the deliverable),
 - has a known method or technique,
 - has well-accepted predecessor and successor steps, and
 - is measurable so that the percentage completed can be determined.

Managing the Information Systems Project: Planning the Project (9/24)

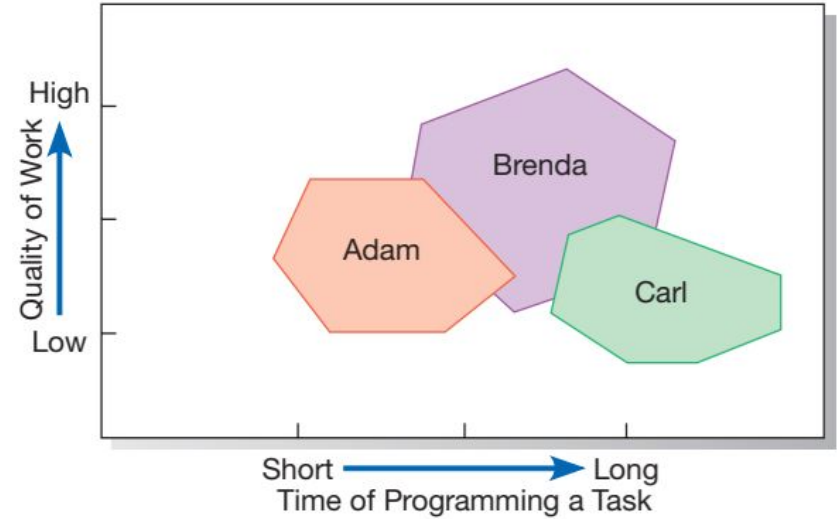
- **Estimating resources and creating a resource plan:** The goal of this activity is to **estimate resource requirements for each project activity** and **to use this information to create a project resource plan**. The resource plan helps assemble and deploy resources in the most effective manner.
 - For example, you would not want to bring additional programmers onto the project at a rate faster than you could prepare work for them. Project managers use a variety of tools to assist in making estimates of project size and costs. The most widely used method is called **COCOMO (COConstructive COst MOdel)**, which uses parameters that were derived from prior projects of differing complexity. COCOMO uses these different parameters to predict human resource requirements for basic, intermediate, and very complex systems.

Managing the Information Systems Project: Planning the Project (10/24)

- **People are the most important and expensive part of project resource planning.**
 - Project time estimates for task **completion** and overall **system quality** are **significantly influenced by the assignment of people to tasks.**
 - It is important to **give people tasks that allow them to learn new skills.**
 - It is equally important to **make sure that project members are not “in over their heads” or working on a task that is not well suited to their skills.**
 - Resource estimates may need to be **revised based upon the skills of the actual person (or people) assigned to a particular activity.**

Managing the Information Systems Project: Planning the Project (11/24)

- The **relative programming speed versus the relative programming quality of three programmers.**
 - The figure suggests that Carl should not be assigned tasks in which completion time is critical and that Brenda should be assigned tasks in which high quality is most vital.



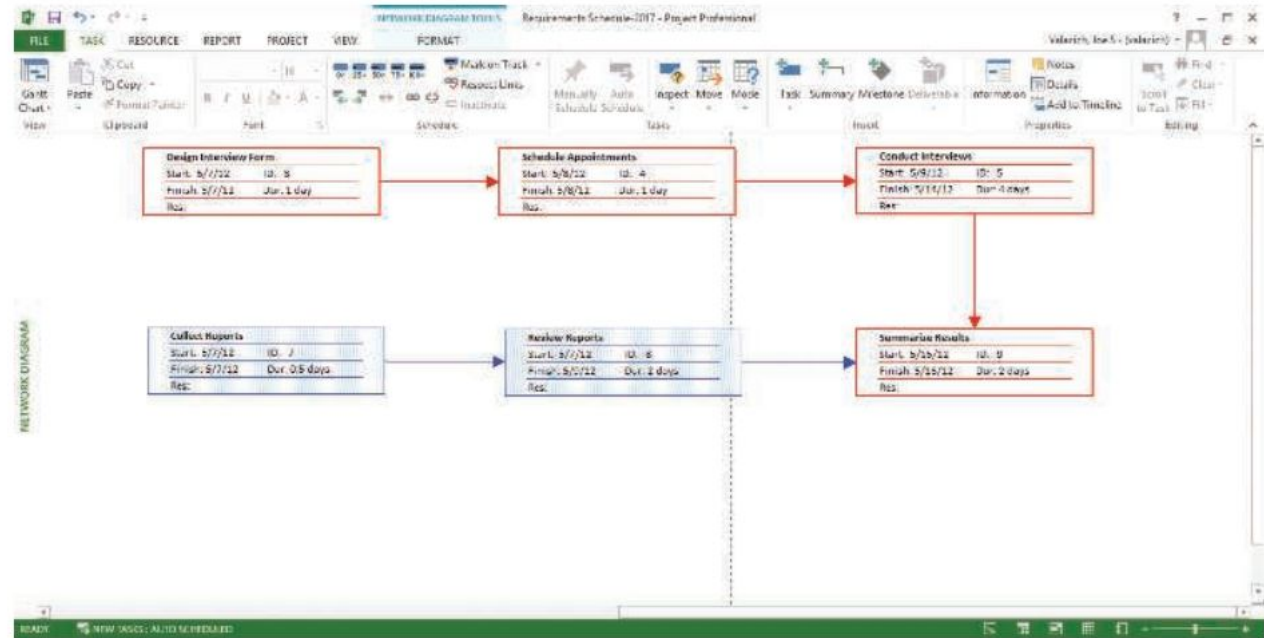
Managing the Information Systems Project: Planning the Project (12/24)

- **One approach to assigning tasks is to assign a single task type (or only a few task types) to each worker for the duration of the project.**
 - For example, you could assign one worker to create all computer displays and another to create all system reports. **Such specialization ensures that both workers become efficient at their own particular tasks.** A worker may **become bored if the task is too specialized or lengthy**, so you could **assign workers to a wider variety of tasks.**
- **However, this approach may lead to lowered task efficiency.**
 - A middle ground would be to **make assignments with a balance of both specialization and task variety.**
- **Assignments depend on the size of the development project and the skills of the project team.**
 - Regardless of the manner in which you assign tasks, **make sure that each team member works only on one task at a time.**
 - **Exceptions** to this rule can occur **when a task occupies only a small portion of a team member's time** (e.g., testing the programs developed by another team member) or **during an emergency.**

Managing the Information Systems Project: Planning the Project (13/24)

- **Developing a preliminary schedule:** During this activity, you use the information on tasks and resource availability to **assign time estimates to each activity** in the work breakdown structure. These time estimates will **enable you to create target starting and ending dates** for the project.
 - Target dates can be **revisited and modified until a schedule is produced that is acceptable to the customer.**
 - **Determining an acceptable schedule** may **require that you find additional or different resources or that the scope of the project be changed.**
 - The schedule may be **represented as a Gantt chart.**
 - A **network diagram** is a graphical depiction of project tasks and their interrelationships. The distinguishing feature of a network diagram is that the ordering of tasks is shown by **connecting tasks—depicted as rectangles or ovals—with their predecessor and successor tasks.** However, the relative size of a node (representing a task) or a gap between nodes does not imply the task's duration. Only the individual task items are drawn on a network diagram, which is why the summary tasks 1, 2, and 6—the black bars.

Managing the Information Systems Project: Planning the Project (14/24)



A network diagram illustrates tasks with rectangles (or ovals) and the relationships and sequences of those activities with arrows

(Source: Microsoft Corporation.)

Managing the Information Systems Project: Planning the Project (15/24)

- **Developing a communication plan:** The goal of this activity is to **outline the communication procedures among management, project team members, and the customer**. The communication plan includes:
 - When and how written and oral reports will be provided by the team?
 - How will team members coordinate work?
 - What messages will be sent to announce the project to interested parties?
 - What kinds of information will be shared with vendors and external contractors involved with the project?
- It is important that **free and open communication occur among all parties** with respect to proprietary information and confidentiality with the customer.

Managing the Information Systems Project: Planning the Project (16/24)

- When developing a communication plan, numerous questions must be answered in order to assure that the plan is comprehensive and complete, including the following:
 - Who are the stakeholders for this project?
 - What information does each stakeholder need?
 - When, and at what interval, does this information need to be produced?
 - What sources will be used to gather and generate this information?
 - Who will collect, store, and verify the accuracy of this information?
 - Who will organize and package this information into a document?
 - Who will be the contact person for each stakeholder should any questions arise?
 - What format will be used to package this information?
 - What communication medium will be most effective for delivering this information to the stakeholder?

Managing the Information Systems Project: Planning the Project (17/24)

- **Once these questions are answered for each stakeholder, a comprehensive communication plan can be developed.**
 - In this plan, a **summary of communication documents, work assignments, schedules, and distribution methods will be outlined.**
 - Additionally, a project **communication matrix** can be developed that provides a summary of the overall communication plan. This matrix can be **easily shared among team members and verified by stakeholders outside the project team**, so that the right people are getting the right information at the right time and in the right format.

Managing the Information Systems Project: Planning the Project (18/24)

Stakeholder	Document	Format	Team Contact	Date Due
Team Members	Project Status Report	Project Intranet	Juan Kim	First Monday of Month
Management Supervisor	Project Status Report	Hard Copy	Juan Kim	First Monday of Month
User Group	Project Status Report	Hard Copy	James Kim	First Monday of Month
Internal IT Staff	Project Status Report	E-Mail	Jackie James	First Monday of Month
IT Manager	Project Status Report	Hard Copy	Juan Jeremy	First Monday of Month
Contract Programmers	Software Specifications	E-Mail/Project Intranet	Jordan Kim	October 1, 2017
Training Subcontractor	Implementation and Training Plan	Hard Copy	Jordan James	January 7, 2018

The project communication matrix provides a high-level summary of the communication plan

Managing the Information Systems Project: Planning the Project (19/24)

- **Identifying and assessing risk:** The goal of this activity is to **identify sources of project risk and estimate the consequences of those risks.**
Risks might arise from
 - the use of new technology,
 - prospective users' resistance to change,
 - availability of critical resources,
 - competitive reactions or changes in regulatory actions due to the construction of a system, or
 - team member inexperience with technology or the business area.
- You should continually try to identify and assess project risk.
- Although we list the **identification of risks and the outline of project scope as two discrete activities**, they are highly related and often concurrently discussed.

Managing the Information Systems Project: Planning the Project (20/24)

- **Creating a preliminary budget:** During this phase, you need to create a preliminary budget that outlines the planned expenses and revenues associated with your project. The project justification will demonstrate that the benefits are worth these costs.
- Figure next shows a **cost-benefit analysis** for a new development project.
 - This analysis shows **net present value calculations of the project's benefits and costs** as well as a **return on investment and cash flow analysis**.

Managing the Information Systems Project: Planning the Project (21/24)

Project Feasibility Analysis-2017 (Compatibility Mode) - Excel

Valacich, Joe S. (valacich)

Economic Feasibility Analysis

	0	1	2	3	4	5	TOTALS
Build New System	\$0	\$85,000	\$85,000	\$85,000	\$85,000	\$85,000	
Discount Rate (12%)	1.0000	0.8929	0.7972	0.7118	0.6355	0.5674	
PV of Benefits	\$0	\$75,893	\$67,761	\$60,501	\$54,019	\$48,231	
NPV of Building New System	\$0	\$75,893	\$143,654	\$204,156	\$258,175	\$306,406	\$306,406
One-time COSTS	(\$75,000)						
Continue Maintaining Existing System							
Recurring Costs		(\$35,000)	(\$35,000)	(\$35,000)	(\$35,000)	(\$35,000)	
Discount Rate (12%)	1.0000	0.8929	0.7972	0.7118	0.6355	0.5674	
PV of Recurring Costs	\$0	(\$31,250)	(\$27,902)	(\$24,912)	(\$22,243)	(\$19,860)	
NPV of All COSTS	(\$75,000)	(\$108,250)	(\$134,152)	(\$159,064)	(\$181,307)	(\$201,167)	(\$201,167)
Overall NPV							\$105,239
ROI = Overall NPV / NPV of Costs							62.31%

Break-Even Analysis

	0	1	2	3	4	5
Yearly NPV Cash Flow	(\$75,000)	\$44,643	\$39,860	\$35,589	\$31,776	\$28,371
Overall NPV Cash Flow	(\$75,000)	(\$30,357)	\$9,503	\$45,092	\$76,867	\$105,239

break-even ratio = (yearly NPV cash flow - general NPV / cash flow) / yearly NPV cash flow
Break-even occurs in 1.8 years.

Note: All dollar values have been rounded to the nearest dollar.

A financial cost and benefit analysis for a systems development project
(Source: Microsoft Corporation.)

Managing the Information Systems Project: Planning the Project (22/24)

- **Developing a Project Scope Statement:** An important activity that occurs near the end of the project planning phase is the **development of the Project Scope Statement.**
 - Developed primarily for the customer, this document **outlines work that will be done and clearly describes what the project will deliver.**
 - The project scope statement is **useful to make sure that you, the customer, and other project team members have a clear understanding of the intended project size, duration, and outcomes.**

Managing the Information Systems Project: Planning the Project (23/24)

- **Setting a Baseline Project Plan:** Once all of the prior project planning activities have been completed, you will be **able to develop a Baseline Project Plan**.
 - This baseline plan **provides an estimate of the project's tasks and resource requirements and is used to guide the next project phase—execution**.
 - As new information is acquired during project execution, the baseline plan will **continue to be updated**.

Managing the Information Systems Project: Planning the Project (24/24)

- At the **end of the project planning phase**, a **review of the baseline project plan is conducted to double-check all information in the plan.**
 - As with the project initiation phase, **it may be necessary to modify the plan, which means returning to prior project planning activities before proceeding.**
- You may **submit the plan and make a brief presentation to the project steering committee** at this time. The **committee can endorse the plan, ask for modifications, or determine that it is not wise to continue the project as currently outlined.**

Managing the Information Systems Project: Executing the Project (1/9)

- **Project execution puts the Baseline Project Plan into action.**
 - Within the context of the SDLC, project execution occurs **primarily during the analysis, design, and implementation phases.**
- There are **five key activities** during project execution.

Project Execution

1. Executing the Baseline Project Plan
2. Monitoring Project Progress against the Baseline Project Plan
3. Managing Changes to the Baseline Project Plan
4. Maintaining the Project Workbook
5. Communicating the Project Status

Managing the Information Systems Project: Executing the Project (2/9)

- **Executing the Baseline Project Plan:** As project manager, you **oversee the execution of the baseline plan**. This means that you
 - initiate the execution of project activities,
 - acquire and assign resources,
 - orient and train new team members,
 - keep the project on schedule and
 - ensure the quality of project deliverables.
- This is a **formidable task**, but a task **made much easier through the use of sound project management techniques**.
- For example, as tasks are completed during a project, they can be “**marked**” as **completed on the project schedule**.
- Members of the project team will come and go. You are **responsible for initiating new team members by providing them with the resources they need and helping them assimilate into the team**.
 - You may want to **plan social events, regular team project status meetings, team-level reviews of project deliverables, and other group events to mold the group into an effective team**.

Managing the Information Systems Project: Executing the Project (3/9)

- **Monitoring project progress against the Baseline Project Plan:** While you execute the Baseline Project Plan, you should **monitor your progress**. If the **project gets ahead of (or behind) schedule**, you may have to **adjust resources, activities, and budgets**.
 - Monitoring project activities **can result in modifications to the current plan**.
 - **Measuring the time and effort expended on each activity** will help you improve the accuracy of estimations for future projects.
 - It is possible, with project schedule charts such as **Gantt charts**, to **show progress against a plan**, and it is easy with **network diagrams** to **understand the ramifications of delays in an activity**.
 - Monitoring progress also means that the team leader must
 - evaluate and appraise each team member,
 - occasionally change work assignments or request changes in personnel, and
 - provide feedback to the employee's supervisor.

Managing the Information Systems Project: Executing the Project (4/9)

- **Managing changes to the Baseline Project Plan:** You will encounter **pressure to make changes to the baseline plan**.
 - **Only approved changes** to the project specification can be made and all changes must be reflected in the baseline plan and project workbook, including all charts. That is, a **formal change request must be approved by the steering committee**.
 - The **request should explain why changes are desired and describe all possible effects on prior and subsequent activities, project resources, and the overall project schedule**. This information allows the project steering committee to more easily evaluate the costs and benefits of a significant midcourse change.
- In addition to changes occurring through formal request, **changes may also occur from events outside your control**. In fact, numerous events may initiate a change to the Baseline Project Plan, including the following possibilities:
 - A slipped completion date for an activity
 - A bungled activity that must be redone
 - The identification of a new activity that becomes evident later in the project
 - An unforeseen change in personnel due to sickness, resignation, or termination

Managing the Information Systems Project: Executing the Project (5/9)

- When an event occurs that delays the completion of an activity, you typically have **two choices**:
 - devise a way to get back on schedule or
 - revise the plan.
- **Devising a way to get back on schedule** is the **preferred approach** because no changes to the plan will have to be made. The ability to head off and smoothly work around problems is a critical skill that you need to master.

Managing the Information Systems Project: Executing the Project (6/9)

- **Maintaining the project workbook:** As in all project phases, maintaining complete records of all project events is necessary.
 - The workbook provides the documentation new team members require to assimilate project tasks quickly.
 - It explains why design decisions were made and is a primary source of information for producing all project reports.

Managing the Information Systems Project: Executing the Project (7/9)

- **Communicating the project status:** The project manager is responsible for keeping all stakeholders—system developers, managers, and customers—abreast of the project status. In other words, communicating the project status focuses on the execution of the project communication plan and the response to any ad hoc information requests by stakeholders.
- **A broad variety of methods can be used to distribute information, each with strengths and weaknesses.**
 - Some methods are easier for the information sender but more difficult or less convenient for the receiver. With the maturing digital networks and the Internet, more and more digital communication is being exchanged. Procedures for communicating project activities vary from formal meetings to informal hallway discussions.
 - Some procedures are useful for informing others of the project's status; others are better for resolving issues; and still others are better for keeping permanent records of information and events.
 - **Two types of information are routinely exchanged throughout the project: work results**—the outcomes of the various tasks and activities that are performed to complete the project—and the project plan—the formal comprehensive document that is used to execute the project; it contains numerous items, including the project charter, project schedule, budgets, and risk plan..

Managing the Information Systems Project: Executing the Project (8/9)

- The table lists numerous communication procedures, their level of formality, and their most likely use. **Whichever procedure you use, frequent communication helps to ensure project success.**

Project Team Communication Methods		
Procedure	Formality	Use
Project workbook	High	Inform Permanent record
Meetings	Medium to high	Resolve issues
Seminars and workshops	Low to medium	Inform
Project newsletters	Medium to high	Inform
Status reports	High	Inform
Specification documents	High	Inform Permanent record
Minutes of meetings	High	Inform Permanent record
Bulletin boards	Low	Inform
Memos	Medium to high	Inform
Brown bag lunches	Low	Inform
Hallway discussions	Low	Inform Resolve issues

Managing the Information Systems Project: Executing the Project (9/9)

- This section outlined your role as the project manager during the execution of the Baseline Project Plan.
 - The ease with which the project can be managed is significantly influenced by the quality of prior project phases.
 - If you develop a high-quality project plan, it is much more likely that the project will be successfully executed.

Managing the Information Systems Project: Closing Down the Project (1/6)

- **The focus of project closedown is to bring the project to an end.**
- Projects can conclude with
 - A **natural termination** occurs when the **requirements of the project have been met**—the project has been completed and is a success.
 - An **unnatural termination** occurs when the **project is stopped before completion**. Several events can cause an unnatural termination of a project.
- For example, it may be learned that the assumption used to guide the project proved to be false, that the performance of the systems or development group was somehow inadequate, or that the requirements are no longer relevant or valid in the customer's business environment. The most likely reasons for the unnatural termination of a project relate to running out of time or money, or both.
- Regardless of the project termination outcome, several activities must be performed:
 - **closing down the project,**
 - **conducting post-project reviews,** and
 - **closing the customer contract.**
- Within the context of the **SDLC**, project **closedown occurs after the implementation phase**. The system maintenance phase typically represents an ongoing series of projects, each of which must be individually managed.

Managing the Information Systems Project: Closing Down the Project (2/6)

- The **project closedown activities** that are described more fully in the remainder of this section:

Project Closedown

1. Closing Down the Project
2. Conducting Postproject Reviews
3. Closing the Customer Contract

Managing the Information Systems Project: Closing Down the Project (3/6)

- **Closing down the project:** During closedown, you perform several diverse activities.
 - For example, if you have several team members working with you, **project completion may signify job and assignment changes for some members.**
 - You will likely be required to assess each team member and **provide an appraisal for personnel files and salary determination.**
 - You may also want to **provide career advice to team members, write letters to superiors praising special accomplishments of team members, and send thank-you letters to those who helped but were not team members.**
 - As project manager, you **must be prepared to handle possible negative personnel issues, such as job termination, especially if the project was not successful.**
 - When closing down the project, it is also **important to notify all interested parties that the project has been completed and to finalize all project documentation and financial records so that a final review of the project can be conducted.**
 - You should also **celebrate the accomplishments of the team.** Some teams will hold a party, and each team member may receive memorabilia (e.g., a T-shirt with “I survived the X project”). The goal is to celebrate the team’s effort to bring a difficult task to a successful conclusion

Managing the Information Systems Project: Closing Down the Project (4/6)

- **Conducting post-project reviews:** Once you have closed down the project, **final reviews of the project should be conducted with management and customers.**
- The **objective** of these reviews is to determine
 - the strengths and weaknesses of project deliverables,
 - the processes used to create them, and
 - the project management process.
- It is important that everyone **understands what went right and what went wrong** in order to improve the process for the next project.
- Remember, the **systems development methodology** adopted by an organization is a living guideline that **must undergo continual improvement.**

Managing the Information Systems Project: Closing Down the Project (5/6)

- **Closing the customer contract:** The focus of this final activity is to ensure that all contractual terms of the project have been met.
 - A project governed by a **contractual agreement** is typically **not completed until agreed to by both parties, often in writing.**
 - Thus, it is **imperative** that you **gain agreement from your customer that all contractual obligations have been met** and that further work is either their responsibility or covered under another SSR or contract.

Managing the Information Systems Project: Closing Down the Project (6/6)

- Closedown is a very important activity.
 - **A project is not complete until it is closed, and it is at closedown that projects are deemed a success or failure.**
 - Completion also **signifies the chance to begin a new project** and to **apply what you have learned.**

Representing and Scheduling Project Plans (1/4)

- A project manager has a wide variety of techniques available for depicting and documenting project plans. These planning documents can take the form of **graphical or textual reports**, although **graphical reports have become most popular for depicting project plans**.
- The most commonly used methods are **Gantt charts** and **network diagrams**.
 - Because Gantt charts **do not (typically) show how tasks must be ordered (precedence) but simply show when a task should begin and when it should end**, they are often more useful for depicting relatively simple projects or sub-parts of a larger project, showing the activities of a single worker, or monitoring the progress of activities compared to scheduled completion dates.
 - Recall that a network diagram **shows the ordering of activities by connecting a task to its predecessor and successor tasks**.
 - Sometimes a network diagram is preferable; other times a Gantt chart more easily shows certain aspects of a project.

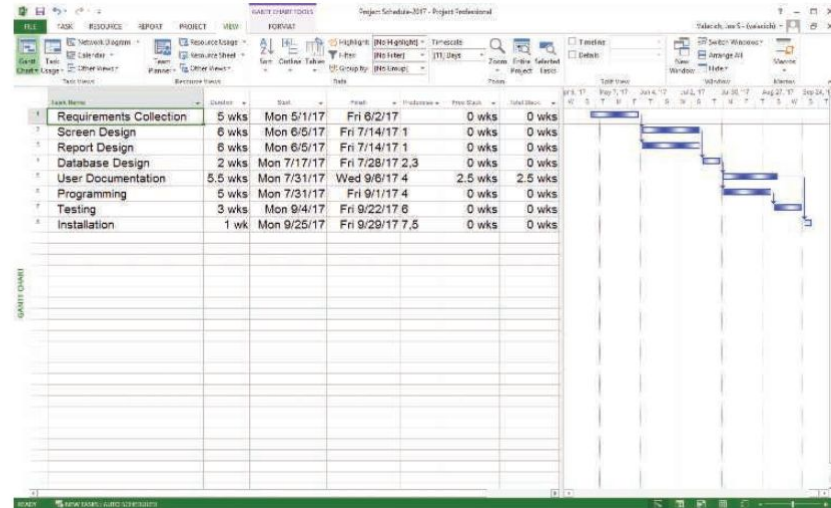
Representing and Scheduling Project Plans (2/4)

- Here are the key differences between these two charts (i.e., Gantt charts and network diagrams):
 - Gantt charts visually show the duration of tasks, whereas a network diagram visually shows the sequence dependencies between tasks.
 - Gantt charts visually show the time overlap of tasks, whereas a network diagram does not show time overlap but does show which tasks could be done in parallel.
 - Some forms of Gantt charts can visually show slack time available within an earliest start and latest finish duration. A network diagram shows this by data within activity rectangles.

Representing and Scheduling Project Plans (3/4)

- Project managers also **use textual reports that depict resource utilization by task, complexity of the project, and cost distributions to control activities.** For example, most project managers use computer-based systems to help develop their graphical and textual reports.

A screen from Microsoft Project for Windows summarizes all project activities, their durations in weeks, and their scheduled starting and ending dates
(Source: Microsoft Corporation.)



Representing and Scheduling Project Plans (4/4)

- A project manager will **periodically review the status of all ongoing project task activities to assess whether the activities will be completed early, on time, or late.**
 - **If early or late, the duration of the activity can be updated.** Once changed, the scheduled start and finish times of all subsequent tasks will also change. Making such a change will also alter a Gantt chart or network diagram used to represent the project tasks.
 - The ability to **easily make changes to a project is a very powerful feature of most project management environments.** It enables the project manager to determine easily how changes in task duration affect the project completion date. It is **also useful for examining the impact of “what if” scenarios of adding or reducing resources, such as personnel, for an activity.**

Using Project Management Software

- **A wide variety of automated project management tools is available to help you manage a development project.**
 - **New versions of these tools are continuously being developed and released** by software vendors. Most of the available tools have **a set of common features that include the ability to define and order tasks, assign resources to tasks, and easily modify tasks and resources.**
 - Project management tools are available to run on IBM-compatible personal computers, the Macintosh, and larger mainframe and workstation-based systems.
- **These systems vary in the number of task activities supported, the complexity of relationships, system processing and storage requirements, and, of course, cost.**
 - Prices for these systems can range from a few hundred dollars for personal computer-based systems to more than \$100,000 for large-scale, multiproject systems.
 - Yet a lot can be done with systems such as Microsoft Project as well as public domain and shareware systems.
 - For example, numerous shareware project management programs (e.g., OpenProj, Bugzilla, and eGroupWare) can be downloaded from the web (e.g., at www.download.com). Because these systems are continuously changing, you should comparison shop before choosing a particular package.

Questions?